QUT Centre for Robotics



ENN583 – Foundations of Robotic Vision

Week 4 – Sem 2 – 2026

Prof Niko Suenderhauf

Local Positioning System Field Demonstrators Wanted for Immediate Start!!!



Role Description: join our team and help with a range of positioning technology field trials across the health, transport, university, council and construction sectors. Full-time or part-time.

Some thinking / troubleshooting involved, but also a lot of straightforward field work (walking around mapping environments and testing the technology).

Required Skillsets:

- Responsibility and maturity (we are working in external client environments like hospitals)
- Reliability and good work ethic
- Some basic familiarity with basic robotics concepts like localization, coding environments and sensors and hardware helpful

Benefits: get experience on what it takes to translate from lab research to deployed commercial technology, and join a great team.

Contact: michael.milford@qut.edu.au with a brief statement why you're interested and attach a CV

LocalPositioningSystem.com

Welcome to Week 4!



CRICOS No.00213J

Welcome to Week 4! – Recap

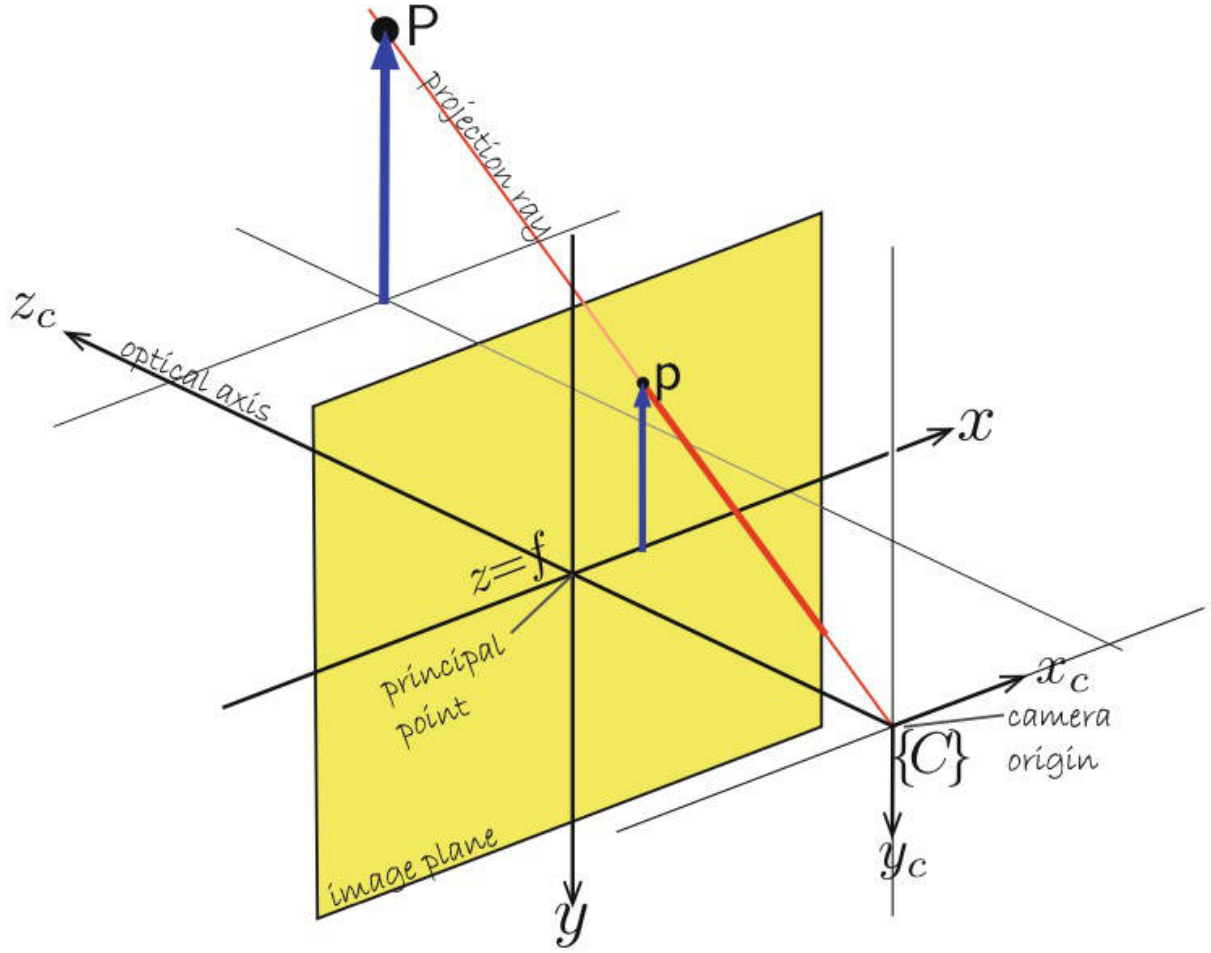
- Week 1:
  - Basic image processing
- Week 2:
  - Interest point detectors, descriptors, matching
- Week 3:
  - Camera models and calibration, Stereo cameras
- **Week 4:**
  - **Multiple View Geometry, Estimating camera movement**

CRICOS No.00213J



Quick Recap: The Central Projection Model (Pinhole Camera Model)

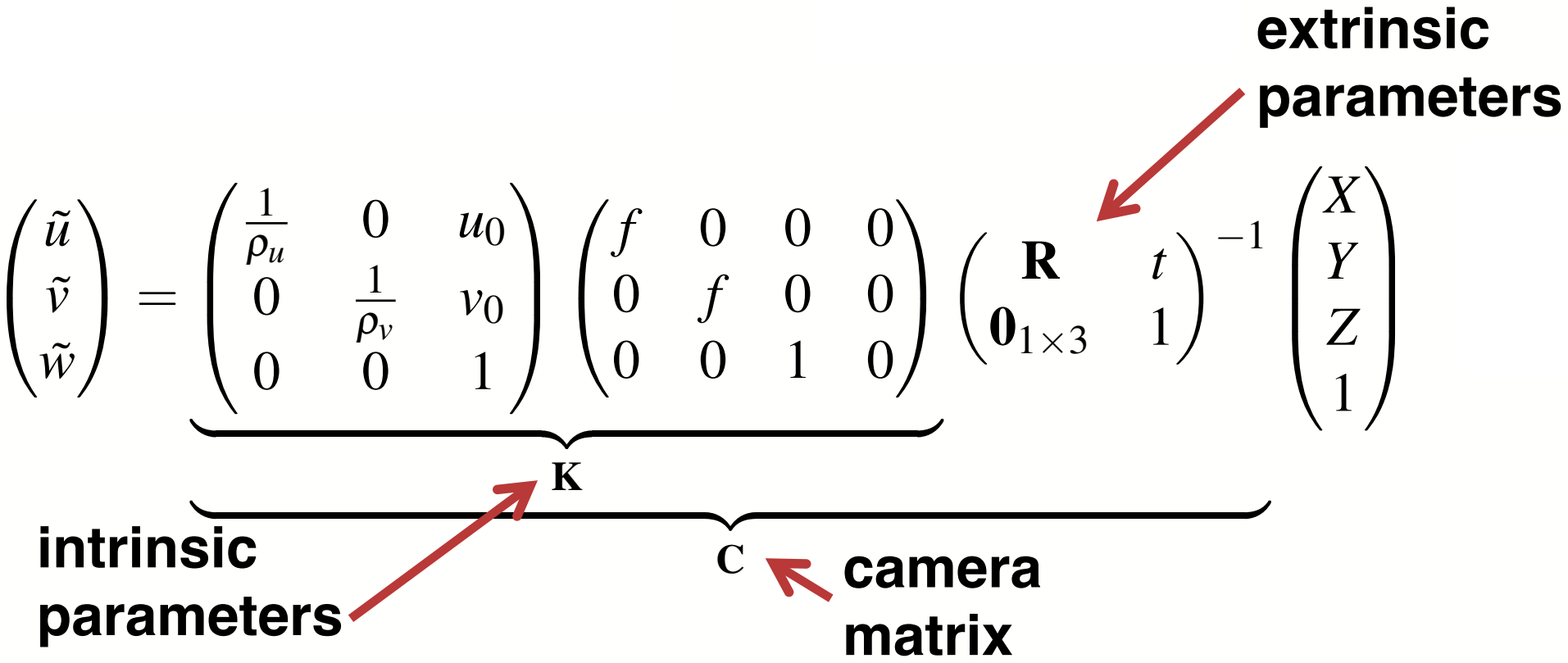
Last Week: The Central Projection Model



Corke, P. (2023). Robotics, Vision and Control: Fundamental Algorithms in Python. Springer Nature.

Corke, P. | Reproduced with permission from Springer Science, & Business Media

Complete camera model



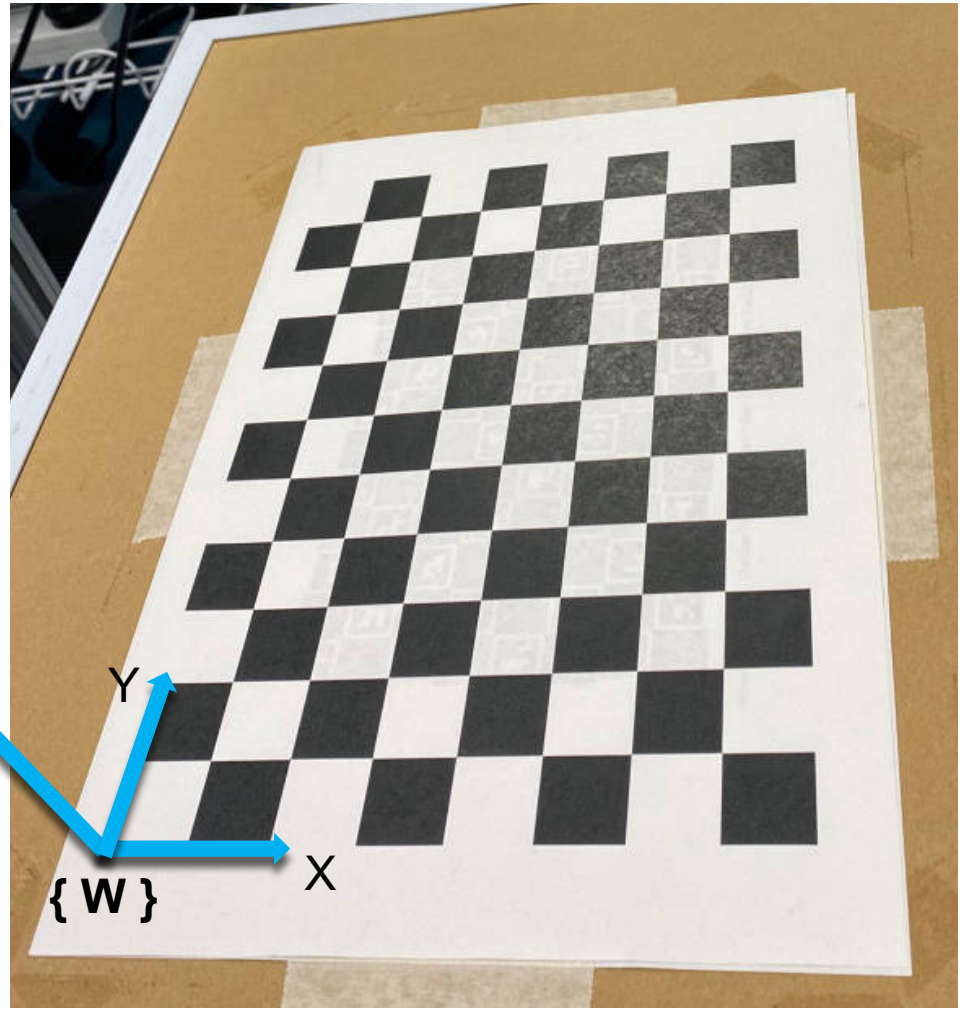
Slide / Illustrations by Peter Corke

Calibration Patterns

Known Unknowns Known

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \cdot \begin{bmatrix} X_w \\ Y_w \\ 1 \end{bmatrix}$$

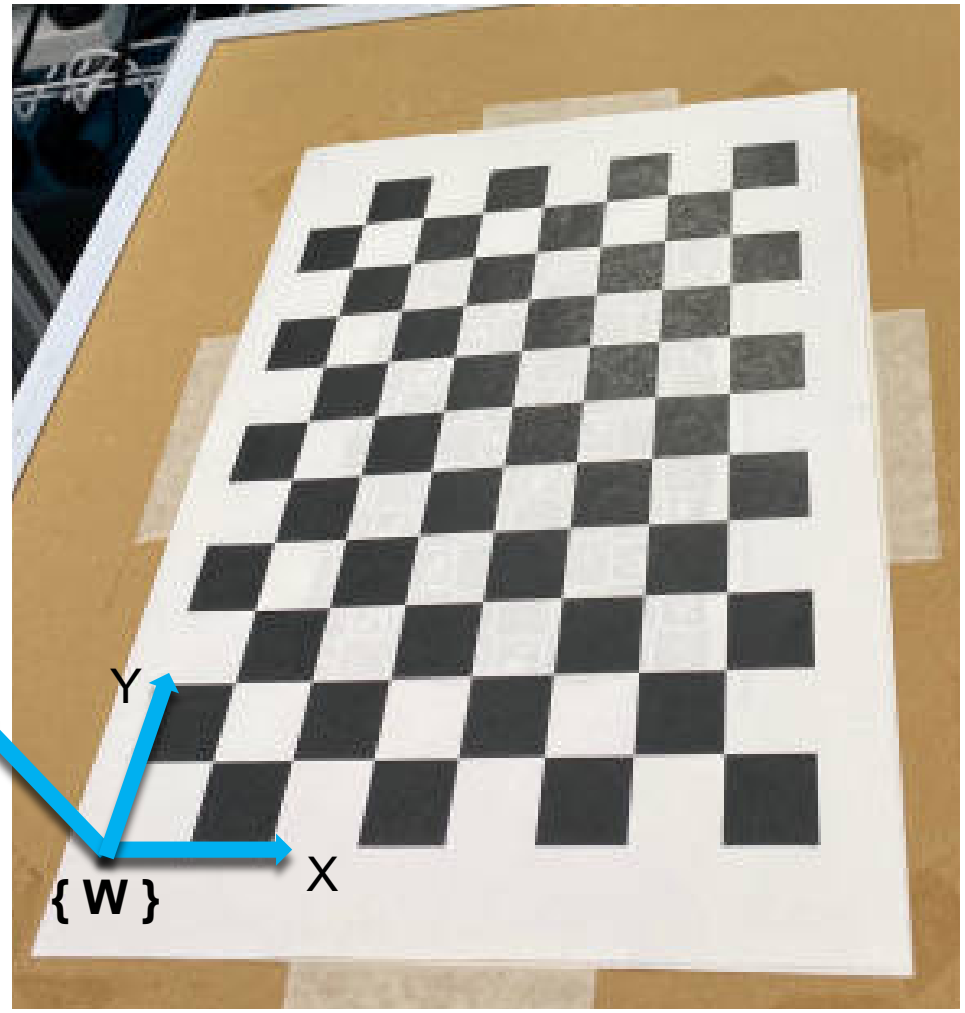
Homography

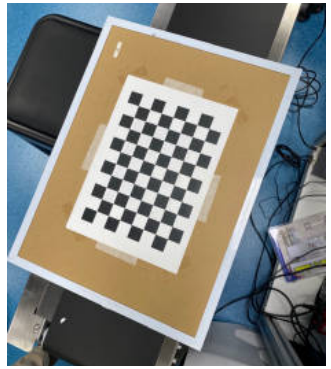
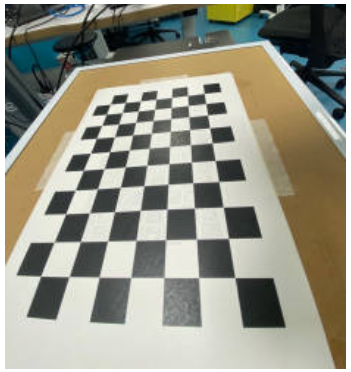
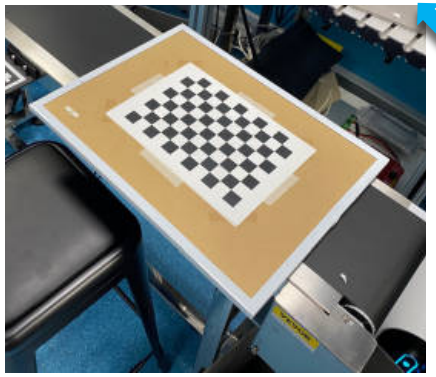


Calibration Patterns

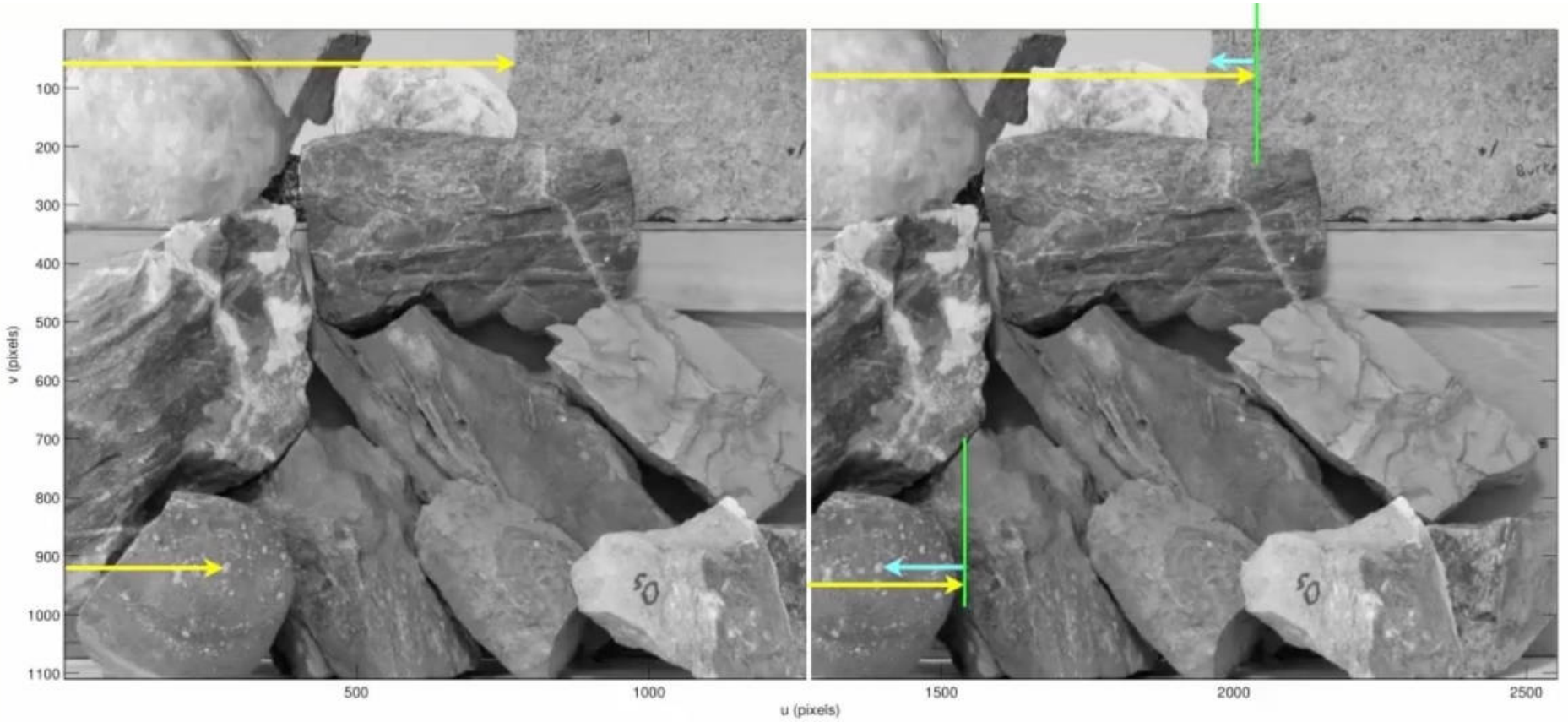
Same Different

$$\begin{bmatrix} h_{11}^j & h_{12}^j & h_{13}^j \\ h_{21}^j & h_{22}^j & h_{23}^j \\ h_{31}^j & h_{33}^j & h_{33}^j \end{bmatrix} = \begin{bmatrix} \alpha_u & 0 & u_0 \\ 0 & \alpha_v & v_0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} r_{11}^j & r_{12}^j & t_1^j \\ r_{21}^j & r_{22}^j & t_2^j \\ r_{31}^j & r_{32}^j & t_3^j \end{bmatrix}$$



Stereo Disparity



- Points in the right image appear shifted to the left
- The shift is less for more distant points

Slide / Illustrations by Peter Corke

From Disparity to 3D

Disparity

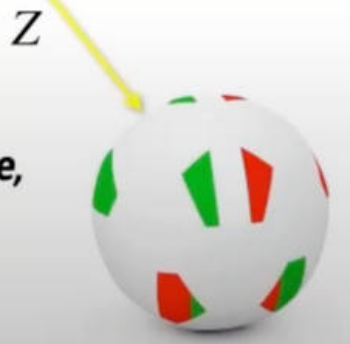
- The horizontal displacement in an image point due to horizontal translation of the camera

$$d \propto \frac{fb}{Z}$$



Kodak Stereo Camera 2008
John Kratz from Bulington NJ, USA | CC BY-SA 2.0 via Wikimedia Commons

- If we know f, b and d then we can estimate Z



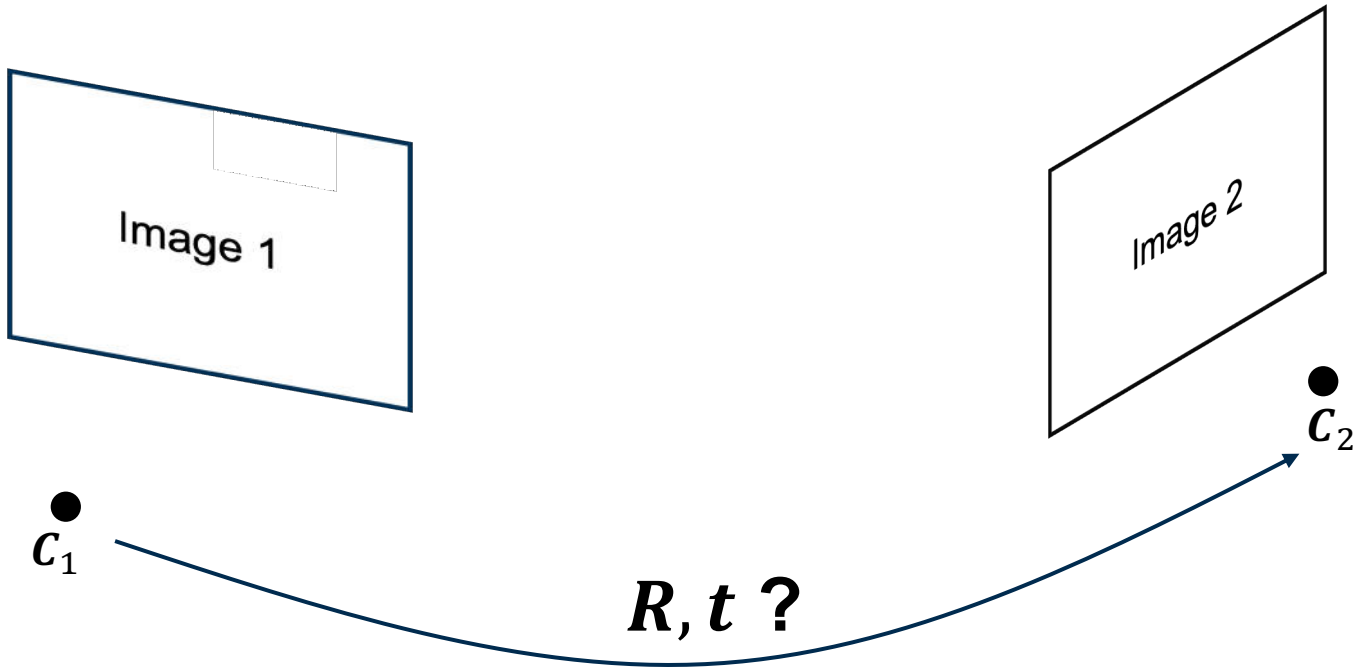
f is focal length, b is baseline, Z is depth.

Slide / Illustrations by Peter Corke



Epipolar Geometry and the Fundamental Matrix

The Setup

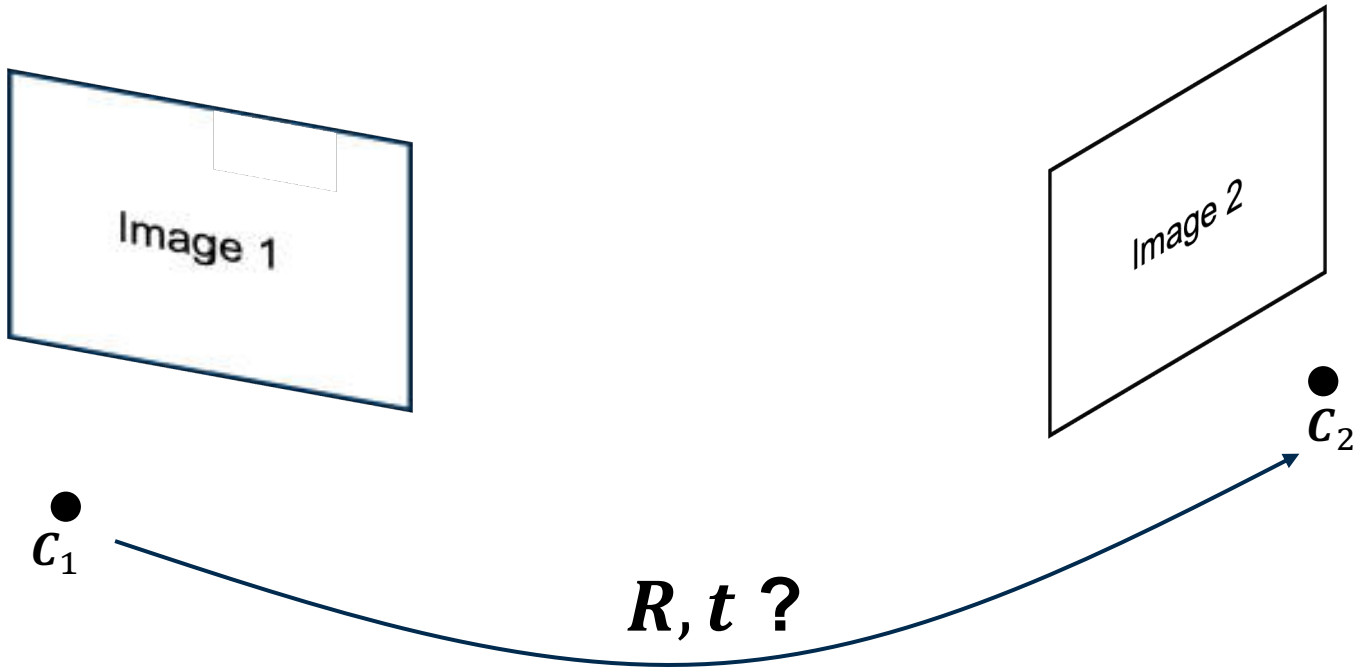


CRICOS No.00213J

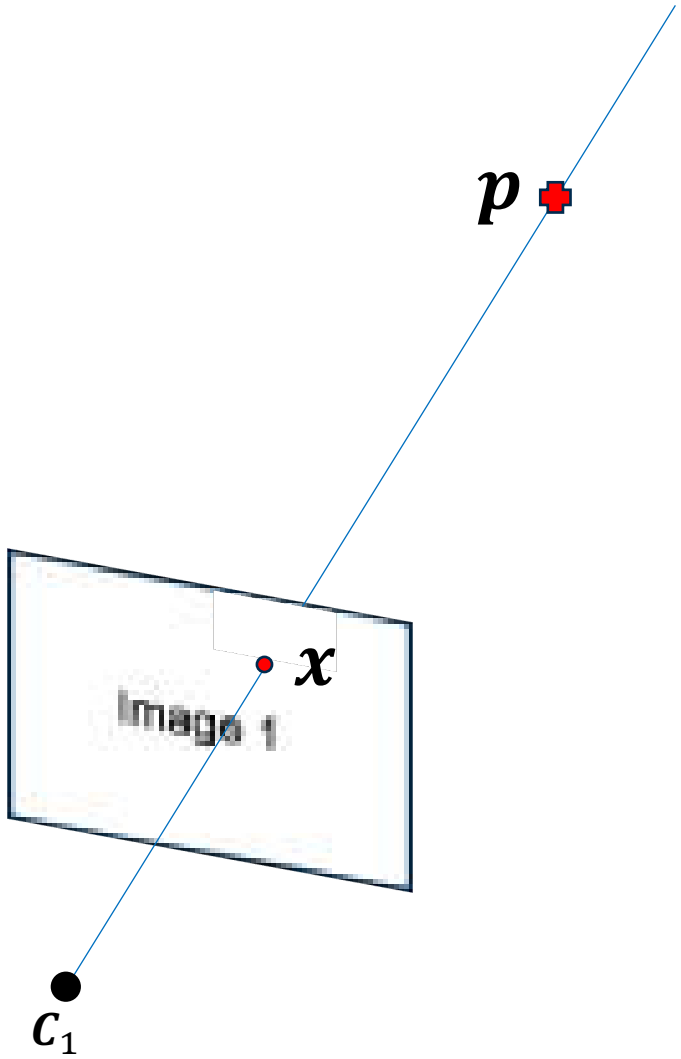
The Setup

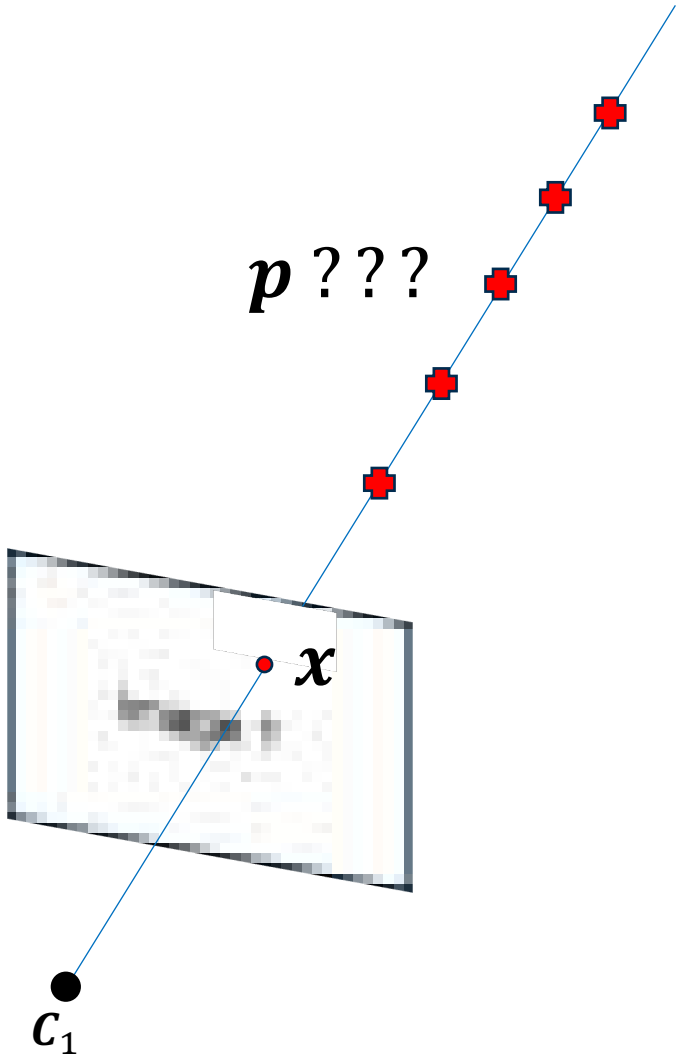
- Two cameras observe a static scene (or the same camera, but it moves).
- We don’t know rotation or translation between the cameras.
- We don’t even know the intrinsic parameters.

- Surprisingly, there is a relation between the points in both images.
- This relation can be captured in a matrix.



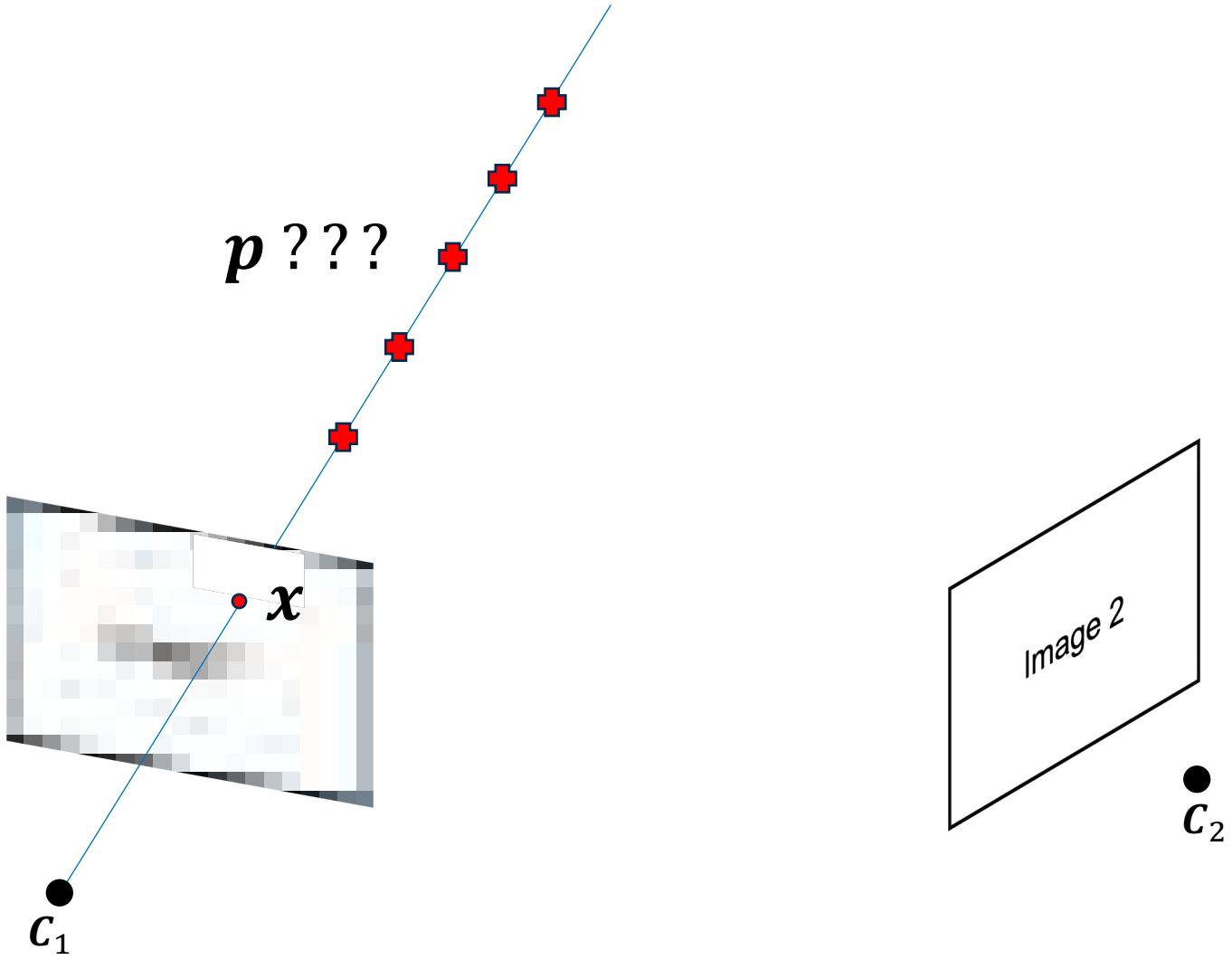
CRICOS No.00213J


p
x
Image 1
c_1

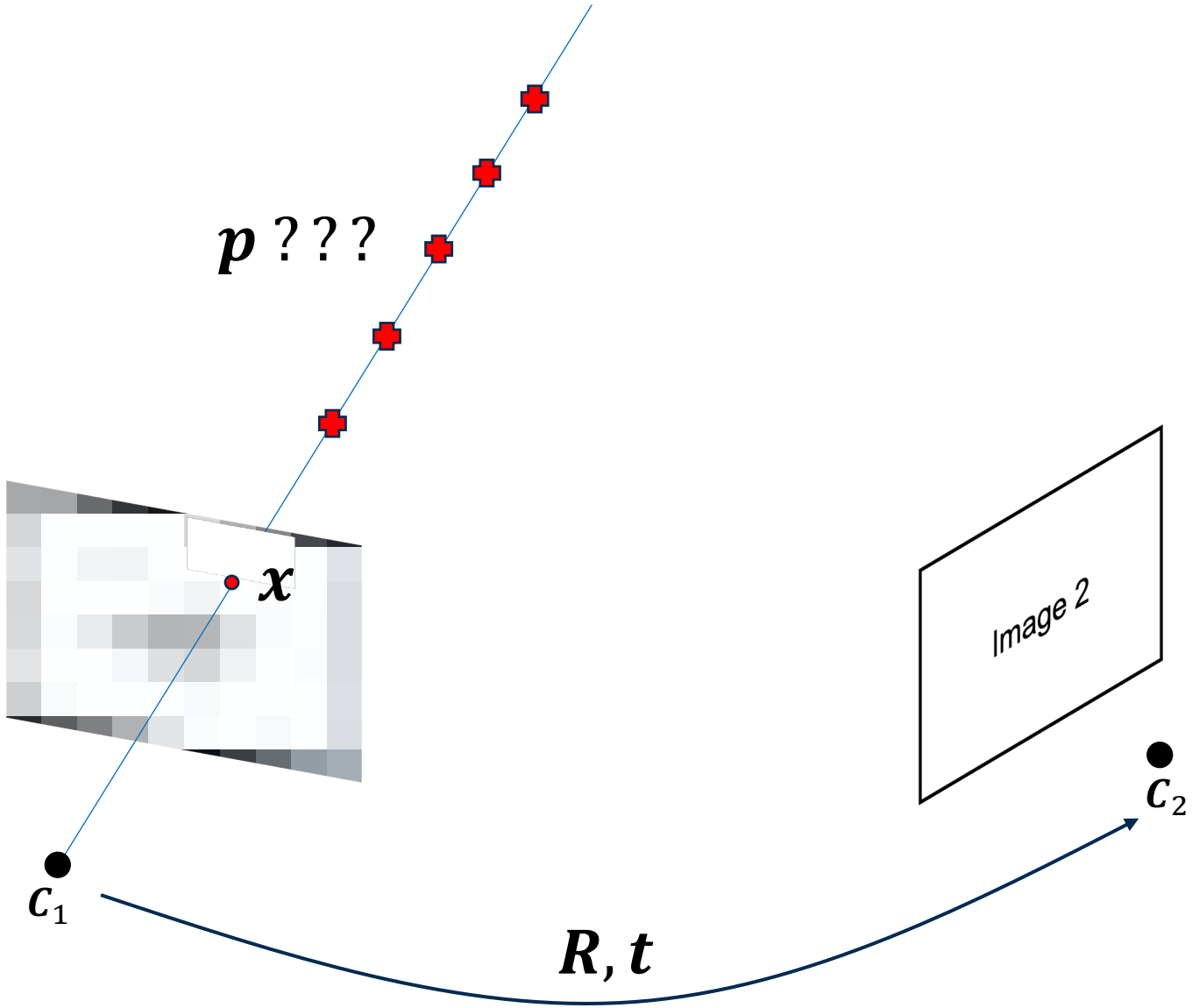


What can we know about **p**?

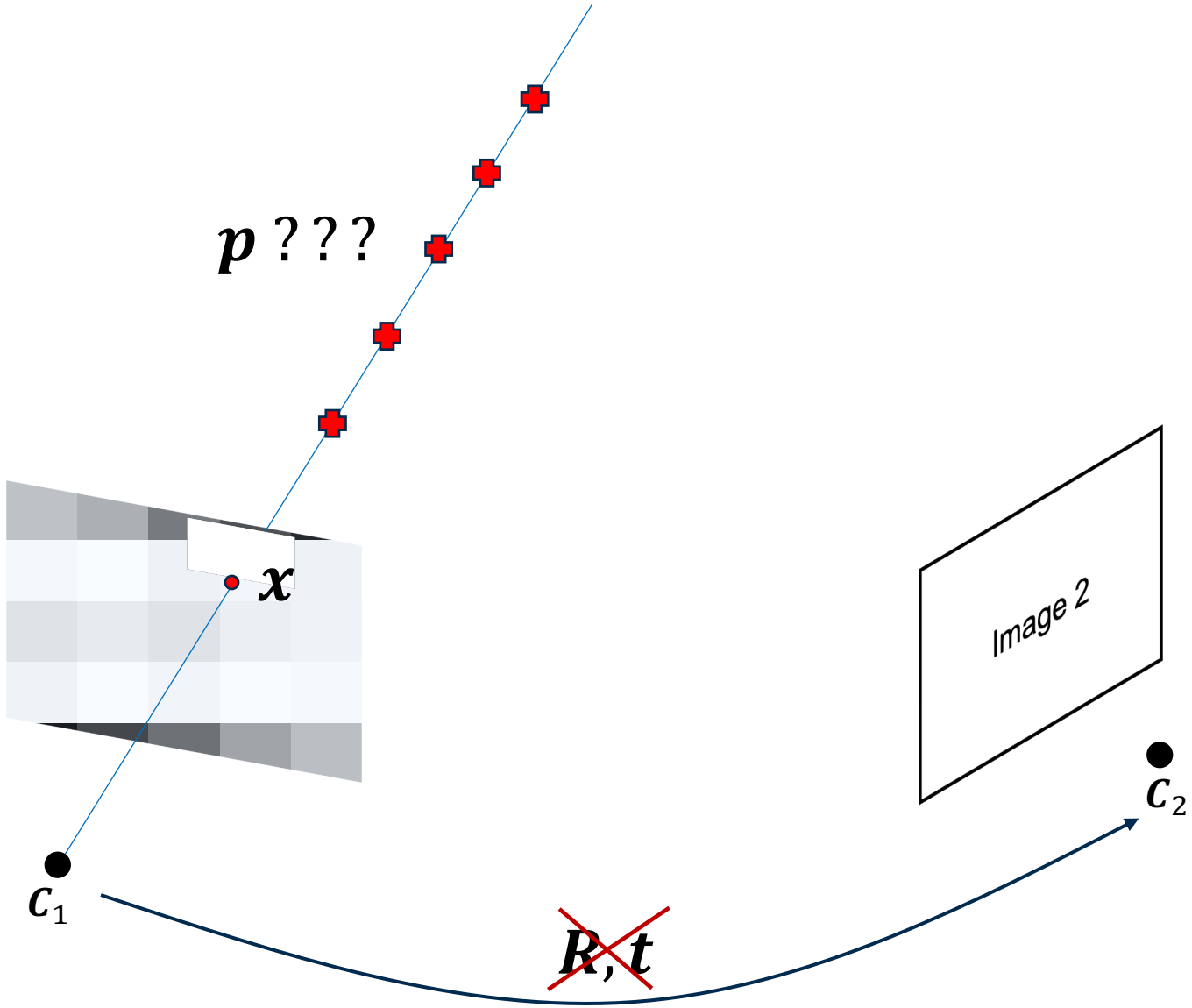
CRICOS No.00213J



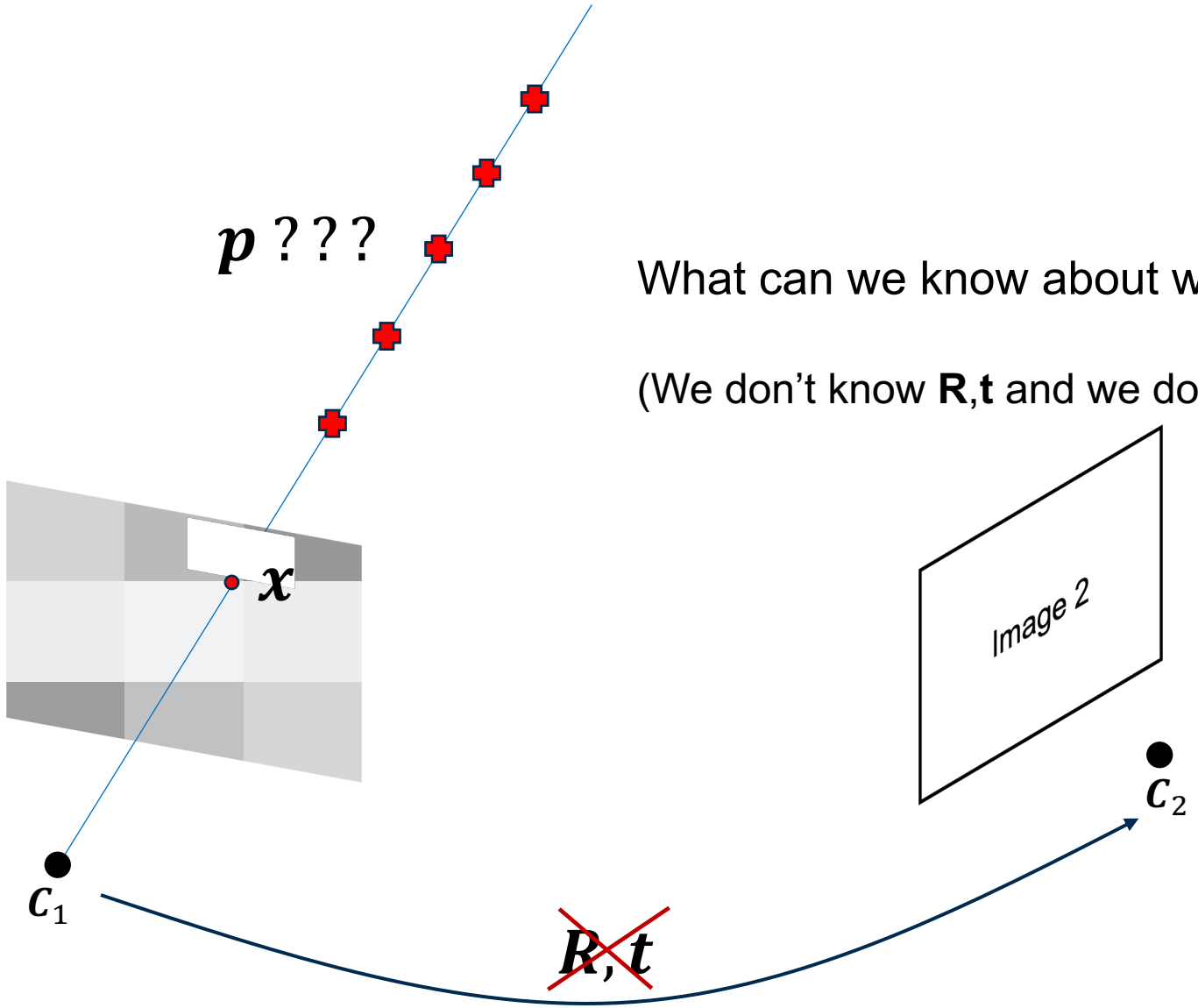
CRICOS No.00213J



CRICOS No.00213J

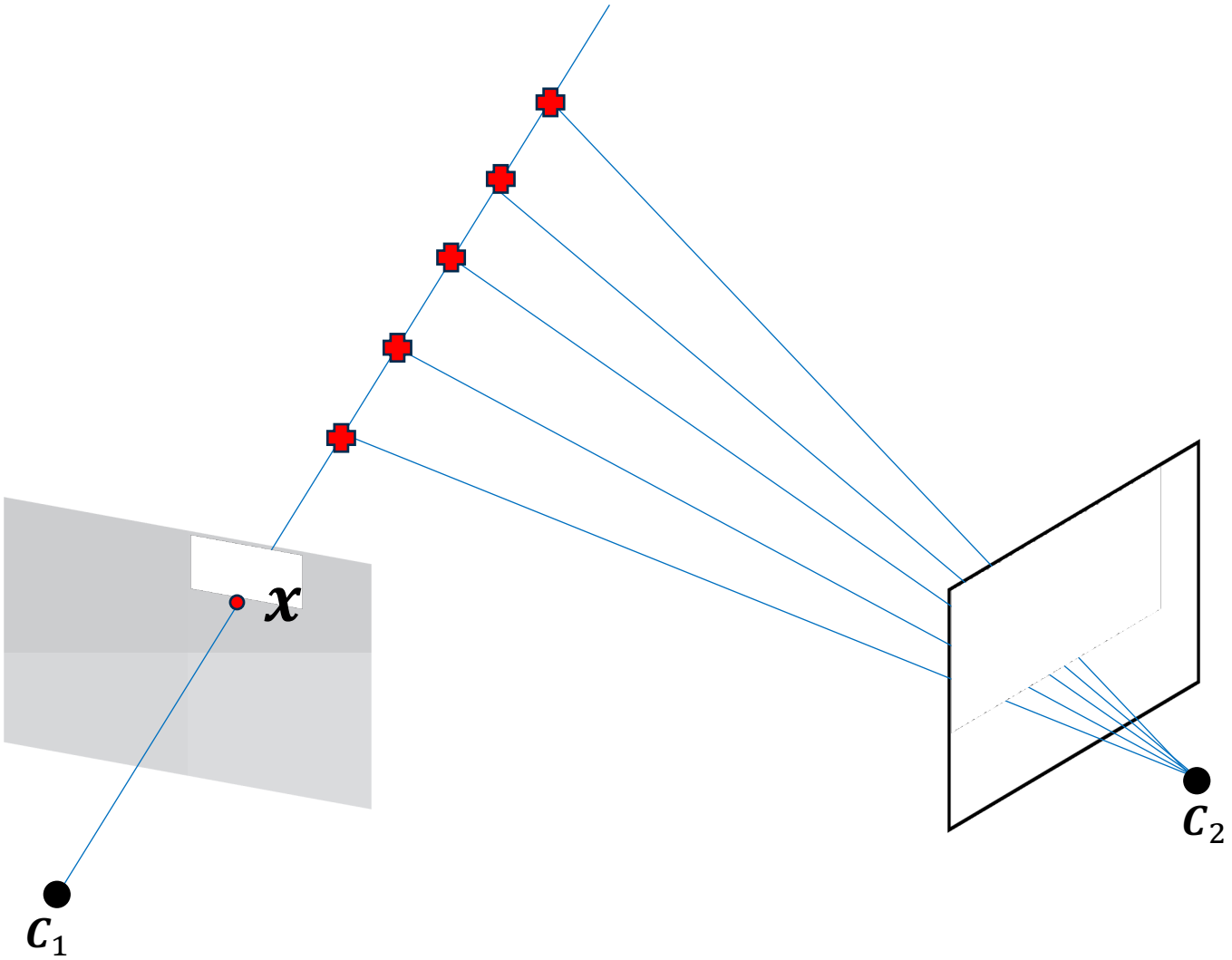


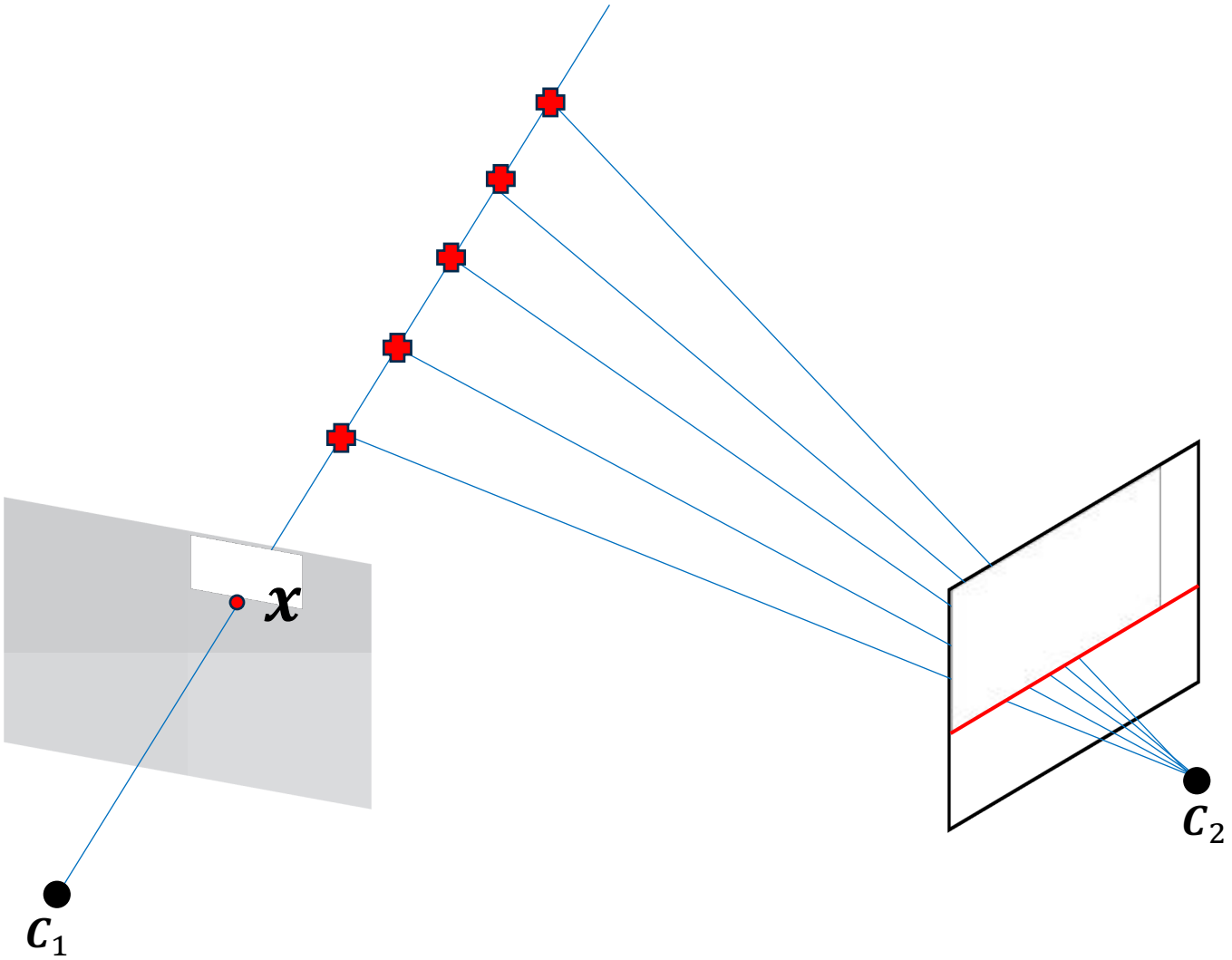
CRICOS No.00213J

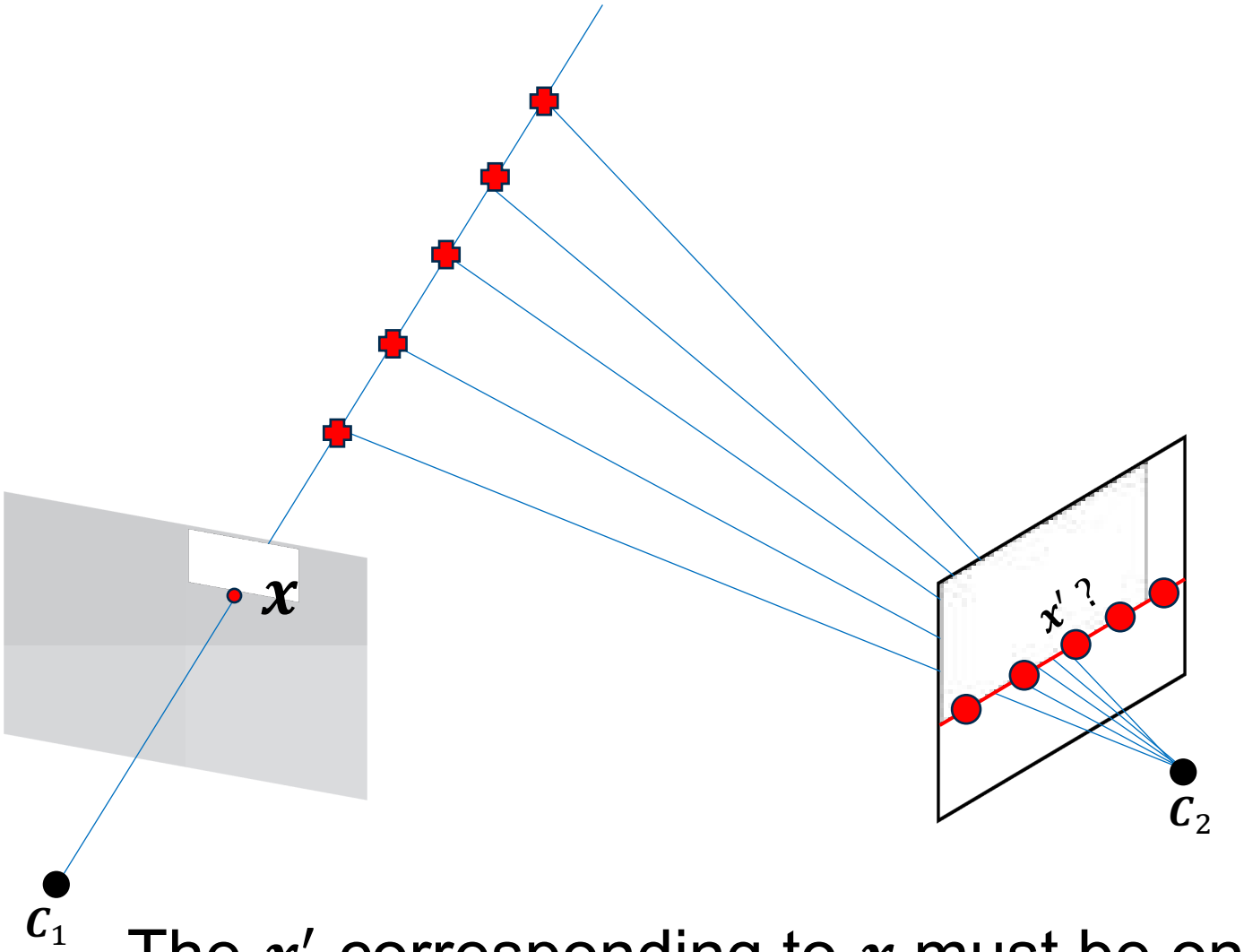


What can we know about where **p** will appear in Image 2?

(We don't know **R**,**t** and we don't even know **K** of the cameras.)


x
C_1
C_2

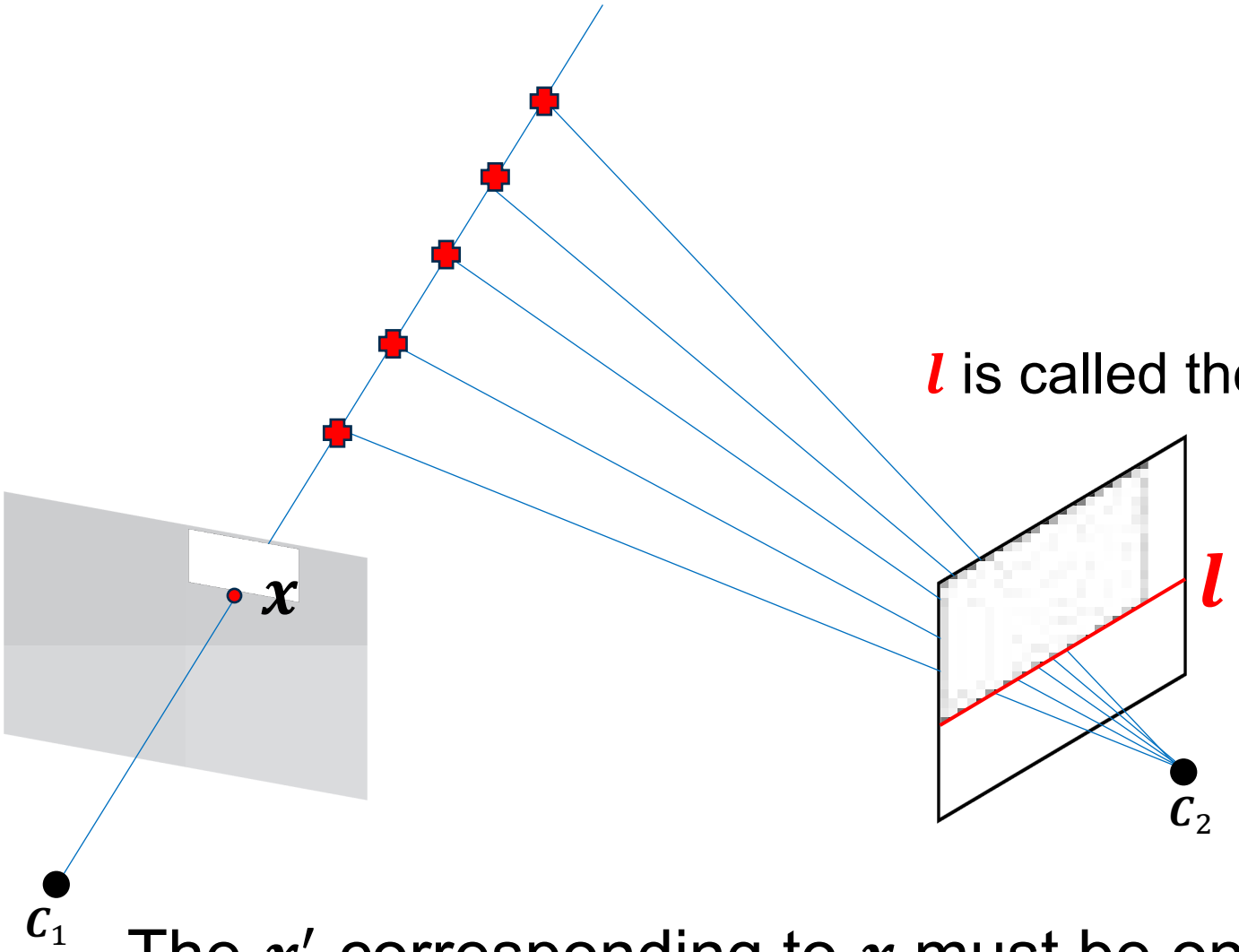

x
C_1
C_2



The x' corresponding to x must be on this line l!

(this is a major insight)

CRICOS No.00213J



l is called the **Epipolar Line** of x.

The x' corresponding to x must be on this line l!

(this is a major insight)

CRICOS No.00213J

Epipolar Geometry

Epipolar geometry describes how a point observed in one camera constrains its matching point in a second camera to lie on an epipolar line.

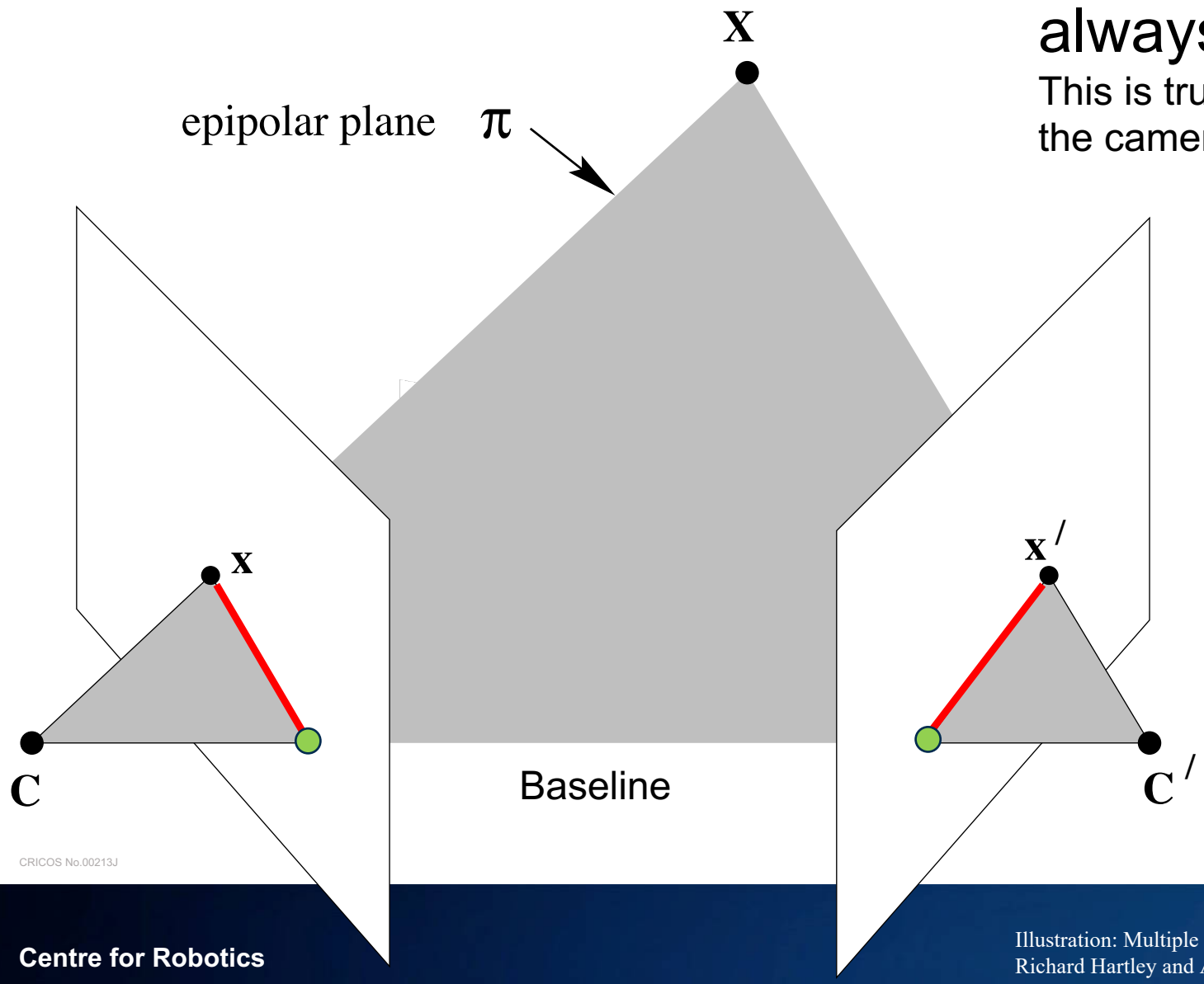
It depends on the cameras' relative geometry, not on the point's unknown depth.

CRICOS No.00213J

Epipolar Geometry

Two cameras and a world point will always form this epipolar geometry.

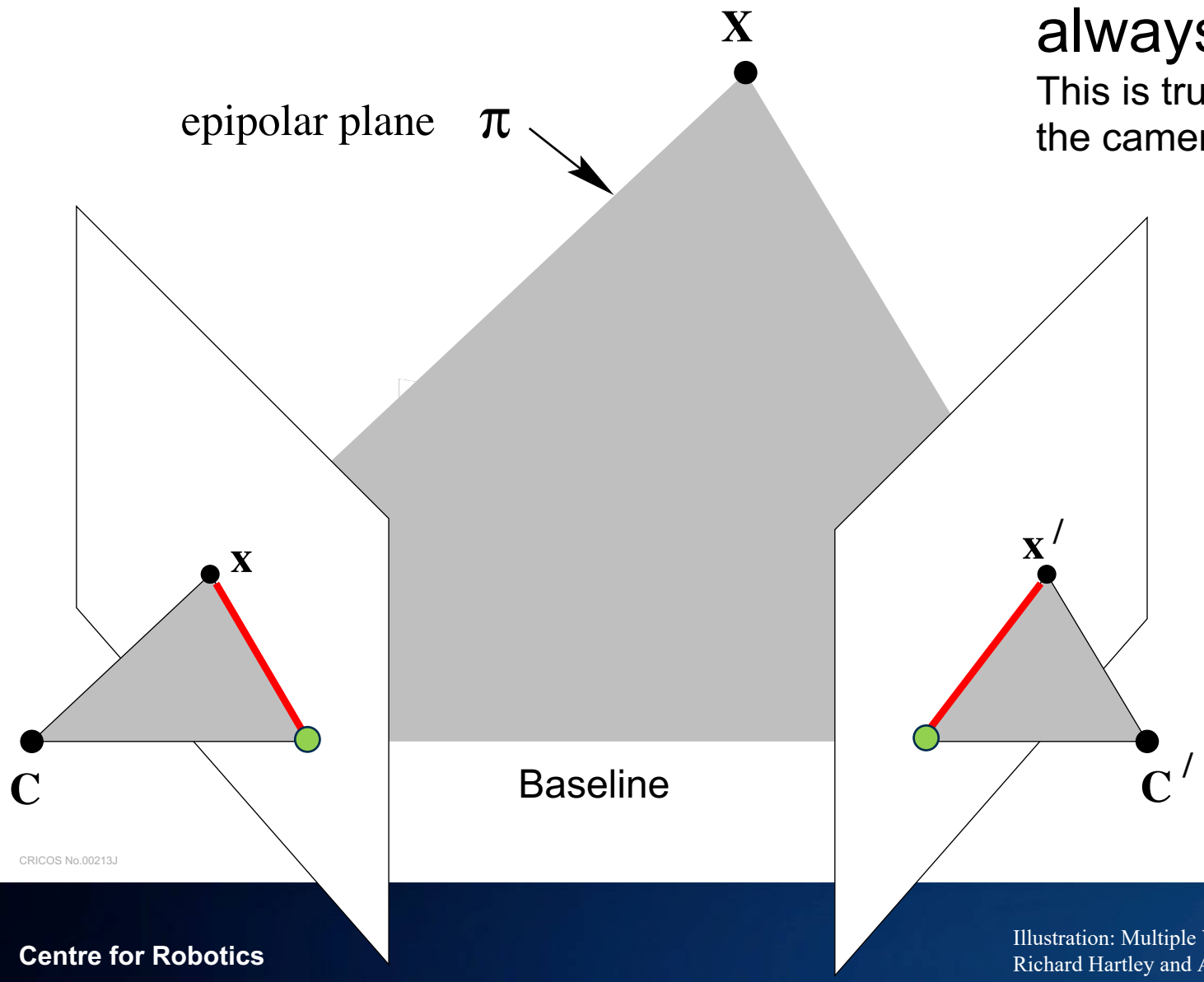
This is true even if we don't know the relative pose of the cameras, or the coordinates of X.



CRICOS No.00213J

Illustration: Multiple View Geometry in Computer Vision.
Richard Hartley and Andrew Zisserman, Cambridge University Press, Second Edition, 2004.

Epipolar Geometry



Two cameras and a world point will always form this epipolar geometry.

This is true even if we don't know the relative pose of the cameras, or the coordinates of X.

The **epipolar plane** is defined by the three points X, C, C'.

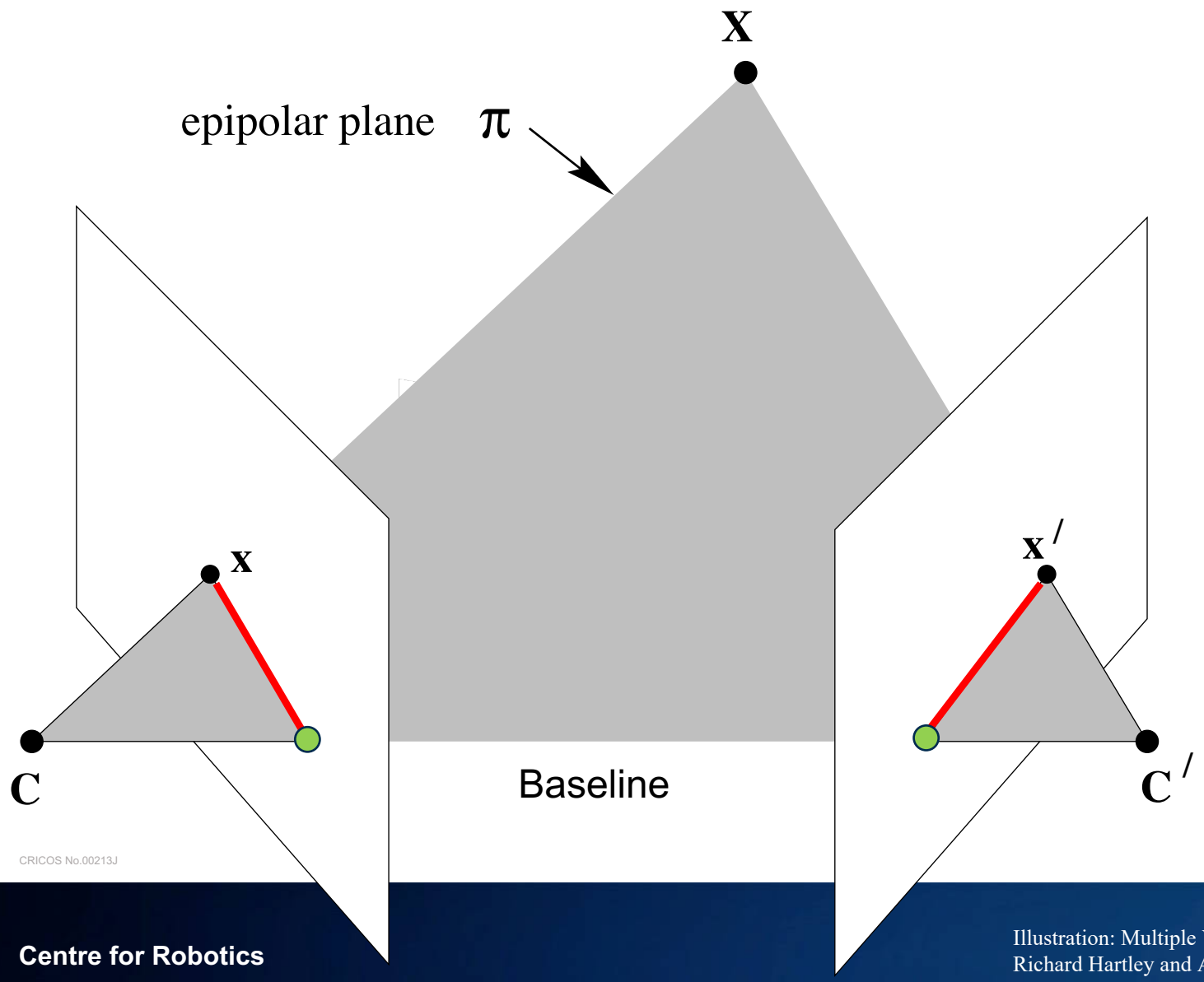
The intersections of the epipolar plane with the images form the **epipolar lines**.

The projection of one camera centre into the other camera is called the **epipole**.
The epipoles are the intersections of the baseline with the images.

CRICOS No.00213J

Illustration: Multiple View Geometry in Computer Vision.
Richard Hartley and Andrew Zisserman, Cambridge University Press, Second Edition, 2004.

Epipolar Geometry

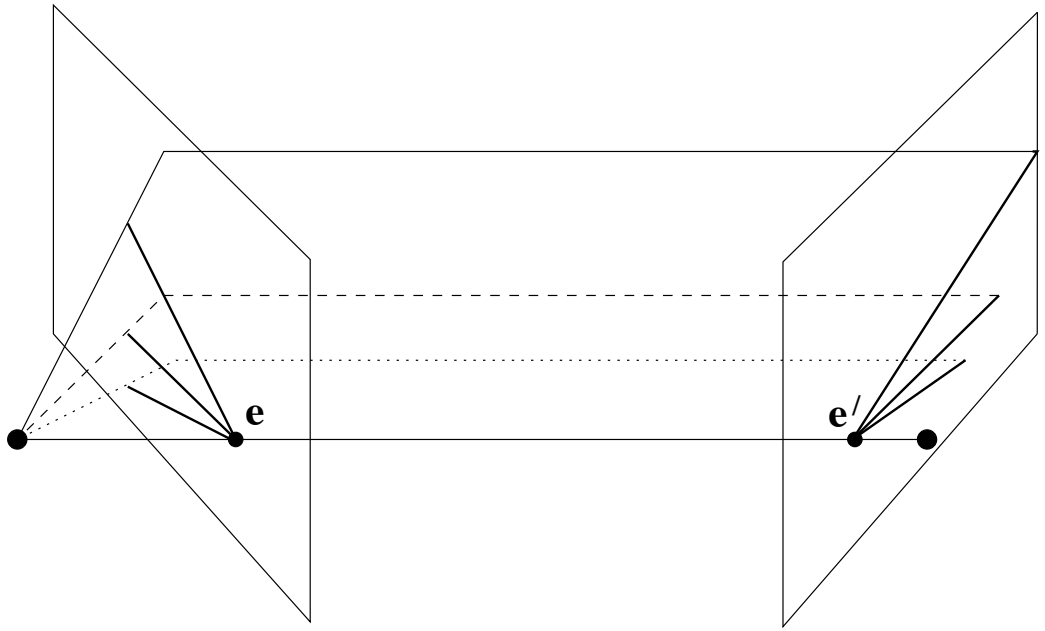


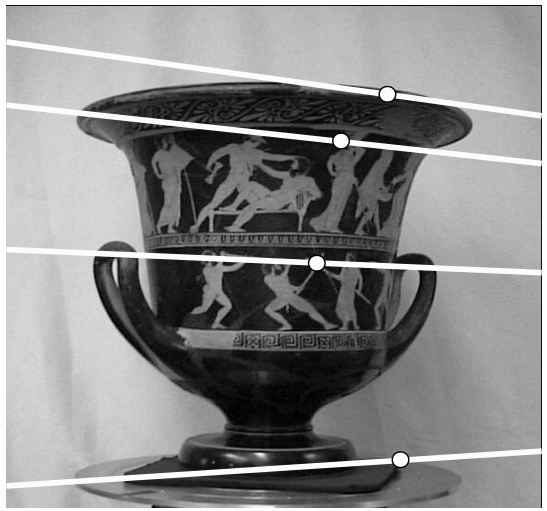
CRICOS No.00213J

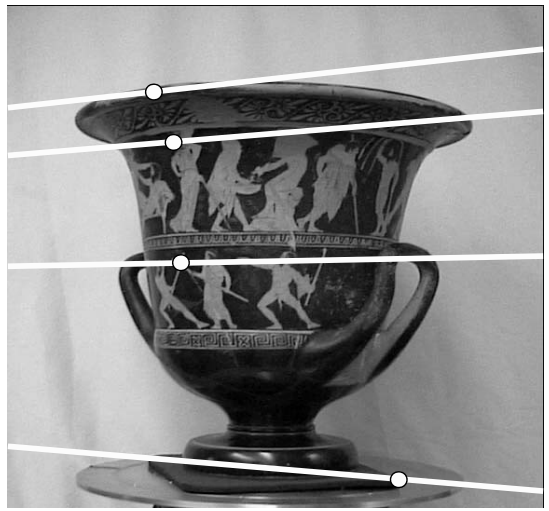
Assume an ideal **stereo camera** system.

- What can we say about the epipolar lines?
- What does that mean for correspondence search like block matching, or disparity calculations?
- Where are the epipoles?

Illustration: Multiple View Geometry in Computer Vision.
Richard Hartley and Andrew Zisserman, Cambridge University Press, Second Edition, 2004.

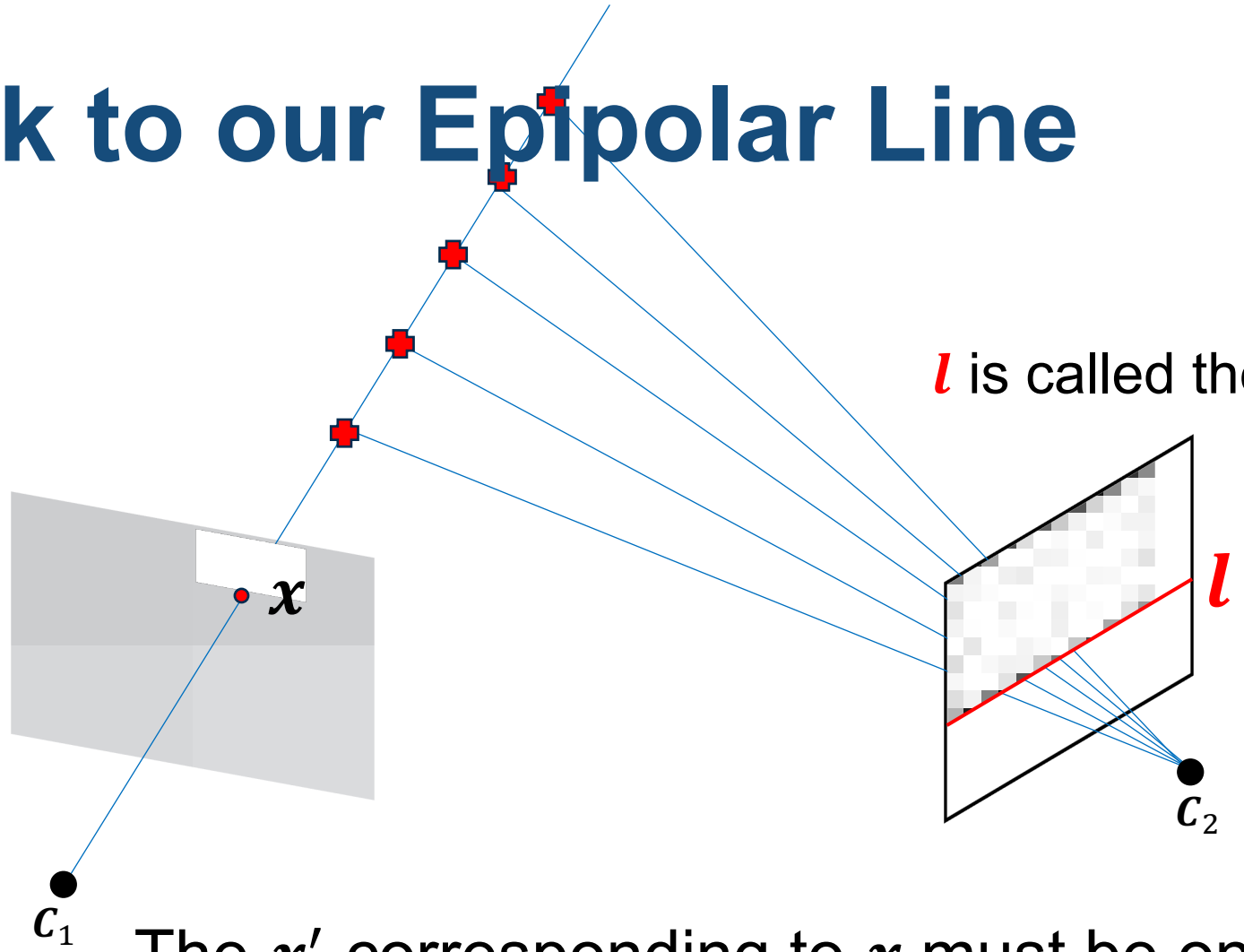






Multiple View Geometry in Computer Vision
Richard Hartley and Andrew Zisserman,
Cambridge University Press, Second Edition, 2004.

Back to our Epipolar Line

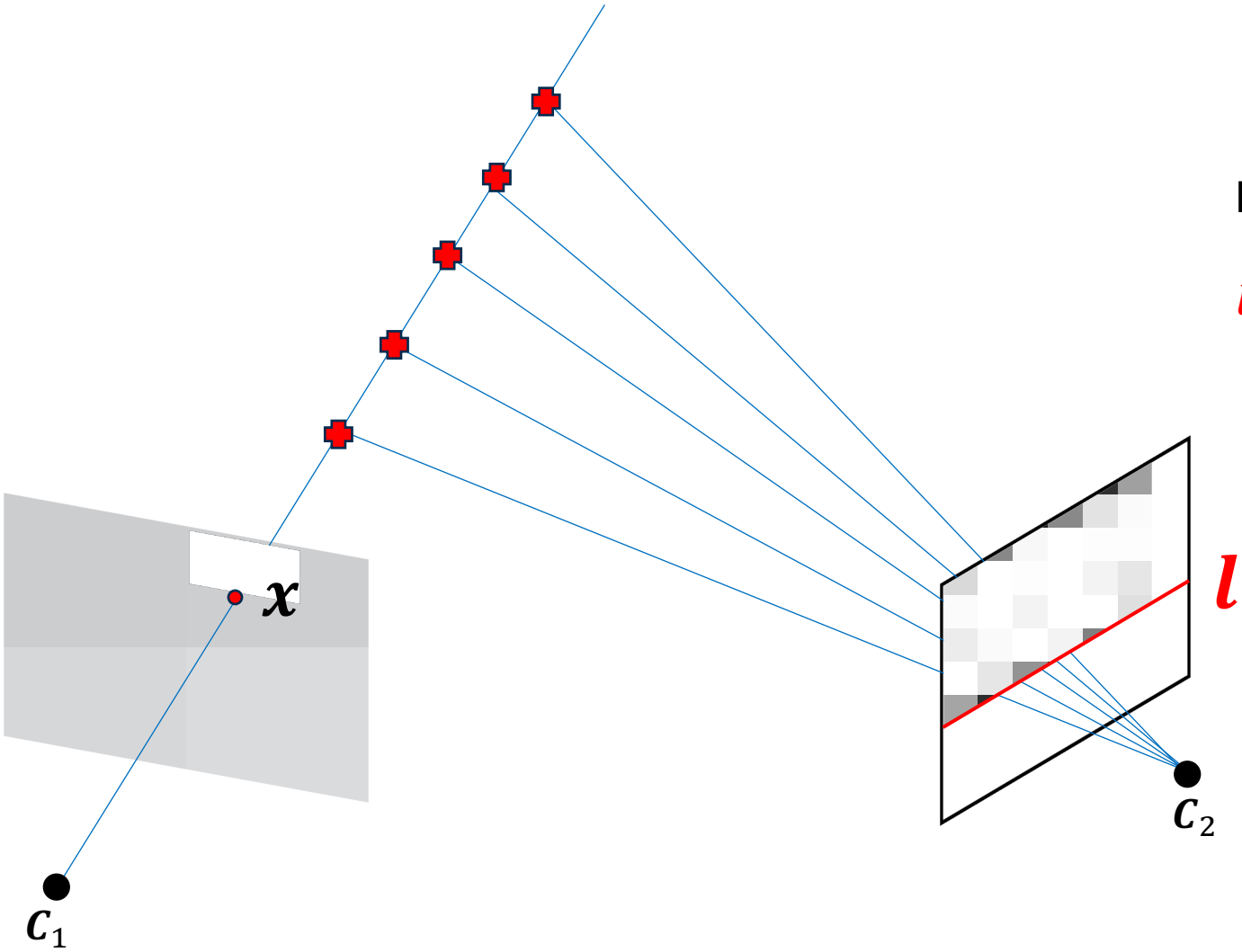


l is called the **Epipolar Line** of x.

Can we calculate or mathematically describe this line?

The x' corresponding to x must be on this line l!

(this is a major insight)



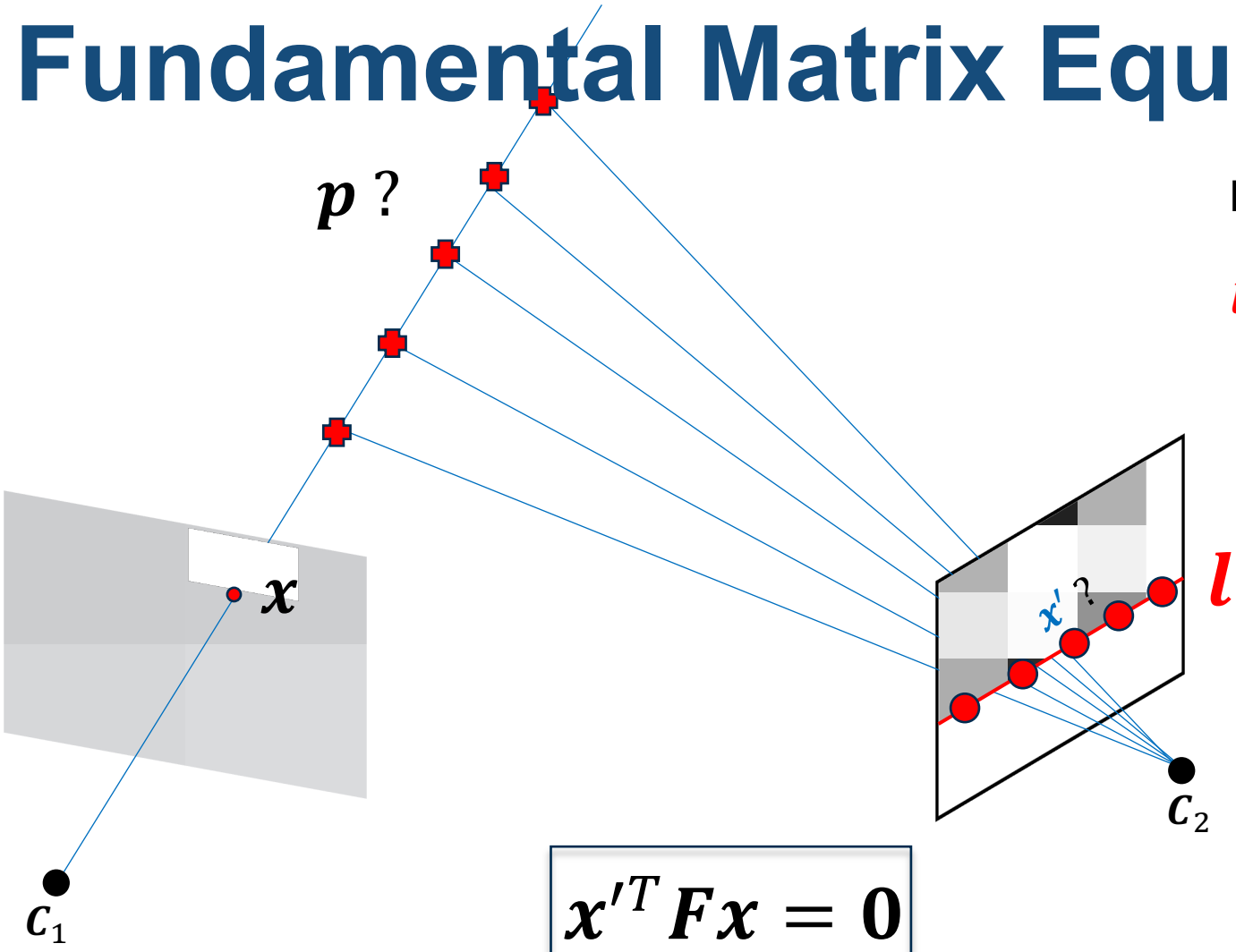
F is the **Fundamental Matrix** (3x3)

l is called the **Epipolar Line** of x.

$$l = Fx$$

CRICOS No.00213J

The Fundamental Matrix Equation



F is the **Fundamental Matrix** (3x3)

l is called the **Epipolar Line** of x.

$$l = Fx$$

x' must be on l!

$$x'^T l = 0$$

Remember from the discussion of homogeneous coordinates!

$$x'^T F x = 0$$

This is true for **any x** and corresponding **x'**!

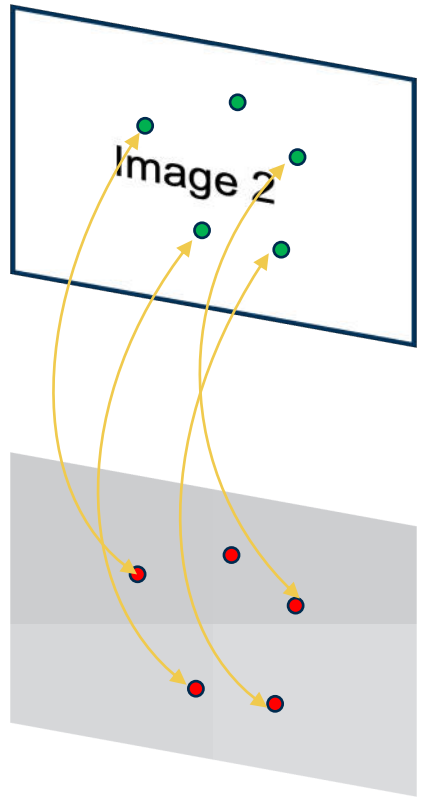
CRICOS No.00213J

Estimating the Fundamental Matrix F

- Need at least **8 2D point correspondences** across both images.
- Use OpenCV's `findHomography()`
  - "8-point algorithm"
  - construct system of linear equations and solve

$$\begin{bmatrix} u_2^i \\ v_2^i \\ 1 \end{bmatrix}^{\mathrm{T}} \mathrm{F} \begin{bmatrix} u_1^i \\ v_1^i \\ 1 \end{bmatrix} = 0$$

$$\begin{bmatrix} u_2^1 u_1^1 & u_2^1 v_1^1 & u_2^1 & v_2^1 u_1^1 & v_2^1 v_1^1 & v_2^1 & u_1^1 & v_1^1 & 1 \\ u_2^2 u_1^2 & u_2^2 v_1^2 & u_2^2 & v_2^2 u_1^2 & v_2^2 v_1^2 & v_2^2 & u_1^2 & v_1^2 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ u_2^n u_1^n & u_2^n v_1^n & u_2^n & v_2^n u_1^n & v_2^n v_1^n & v_2^n & u_1^n & v_1^n & 1 \end{bmatrix} \begin{bmatrix} f_{11} \\ f_{12} \\ f_{13} \\ f_{21} \\ f_{22} \\ f_{23} \\ f_{31} \\ f_{32} \\ f_{33} \end{bmatrix} = 0$$



Maths illustration of equations courtesy of Davide Scaramuzza

Maths illustration of equations courtesy of Davide Scaramuzza

Estimating the Fundamental Matrix F

$$\begin{bmatrix} u_2^1u_1^1 & u_2^1v_1^1 & u_2^1 & v_2^1u_1^1 & v_2^1v_1^1 & v_2^1 & u_1^1 & v_1^1 & 1 \\ u_2^2u_1^2 & u_2^2v_1^2 & u_2^2 & v_2^2u_1^2 & v_2^2v_1^2 & v_2^2 & u_1^2 & v_1^2 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ u_2^nu_1^n & u_2^nv_1^n & u_2^n & v_2^nu_1^n & v_2^nv_1^n & v_2^n & u_1^n & v_1^n & 1 \end{bmatrix} \begin{bmatrix} f_{11} \\ f_{12} \\ f_{13} \\ f_{21} \\ f_{22} \\ f_{23} \\ f_{31} \\ f_{32} \\ f_{33} \end{bmatrix} = 0$$

- This is of the form $\mathbf{A} \cdot \boldsymbol{f} = 0$: Each correspondence contributes one linear constraint on the nine entries of $\boldsymbol{f}$
- There is a trivial solution: $\boldsymbol{f} = 0$, but that is not what we want!
- We have to impose a scale constraint on $\boldsymbol{f}$: $\|\boldsymbol{f}\| = 1$
- Then we solve $\boldsymbol{f}^* = \mathrm{argmin}_{\|\boldsymbol{f}\|=1} \|\boldsymbol{A} \cdot \boldsymbol{f}\|_2^2$, finding the $\boldsymbol{f}$ that comes closest to satisfying all equations.
- A second SVD enforces the required rank-two structure.

Normalise points in image 1 and image 2

↓

Construct A from the normalised correspondences

↓

Compute SVD: $A = U\Sigma V^T$

↓

Take the last row of V^T

↓

Reshape it into a 3×3 matrix

↓

Enforce rank(F) = 2 using another SVD

↓

Denormalise: $F = T'^T \tilde{F} T$

CRICOS No.00213J

Degenerate and Near-Degenerate Cases: When Estimating F Fails

Situation	Main problem
Pure rotation	No translation or meaningful epipoles
Planar scene	Homography is supported, but **F** is not unique
Collinear or clustered points	Insufficient independent constraints
Very small baseline	Translation dominated by noise
Distant points	Weak translational parallax
Forward motion near the epipole	Very small image displacement
Dynamic scene	No single rigid-motion model
Lens distortion or rolling shutter	Pinhole two-view model is violated

- Pure rotation, planar structure and rank-deficient point arrangements are genuine degeneracies.
- Small baselines, distant points and forward motion are usually near-degenerate cases that make estimation poorly conditioned rather than mathematically impossible.

CRICOS No.00213J

Practical Checks for Degeneracy

OpenCV will often return a matrix even for degenerate data.

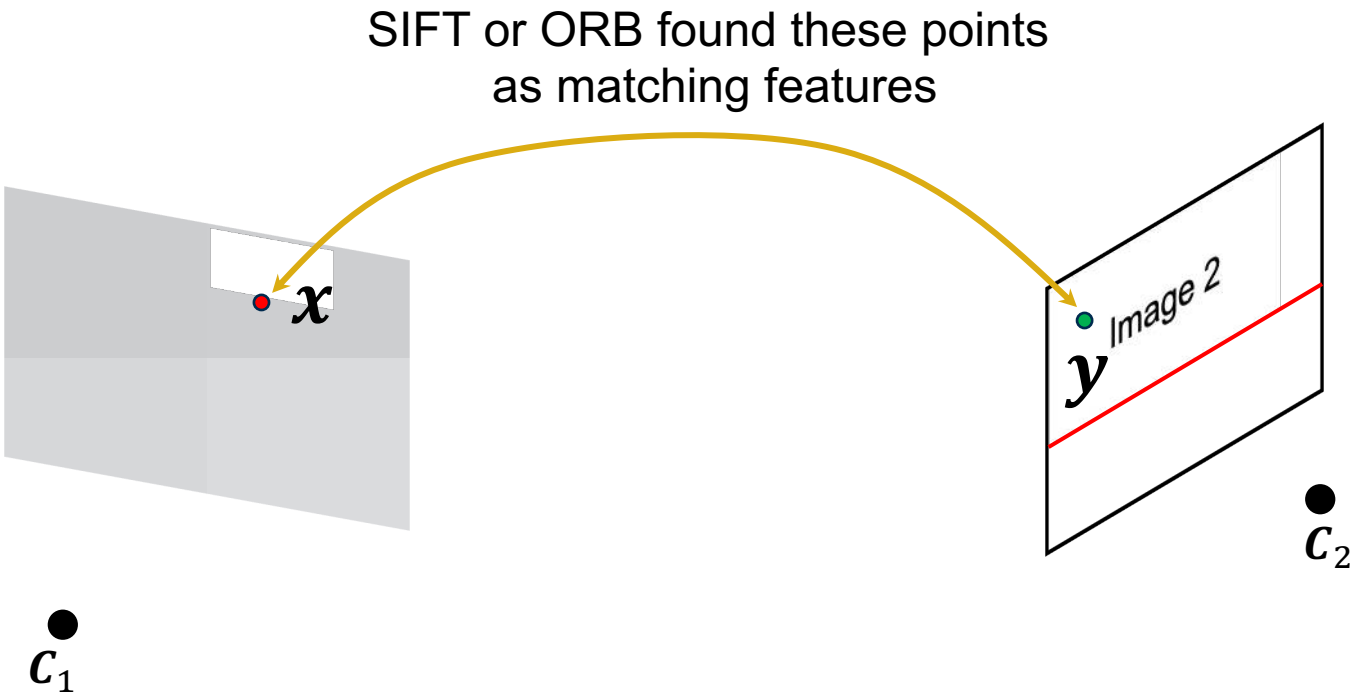
Check	Warning sign	Recommended response
Number of inliers	Too few F-inliers or a very low inlier ratio	Reject the estimate
Spatial distribution	Inliers lie on one line, one plane-like region or a small part of the image	Wait for better-distributed features
Homography comparison	H explains nearly as many points as F	Flag planar scene, pure rotation or weak parallax
Sampson error	Large median error for the F-inliers	Reject or improve feature matching
Cheirality	No clear pose candidate puts most points in front of both cameras	Reject the pose as ambiguous
Resampling stability	Removing a subset of matches causes large changes in R or t	Treat the estimate as poorly conditioned
Temporal consistency	Sudden implausible changes in rotation or translation direction	Reject or fall back to the previous motion model

CRICOS No.00213J

What can we do with the Fundamental Matrix?

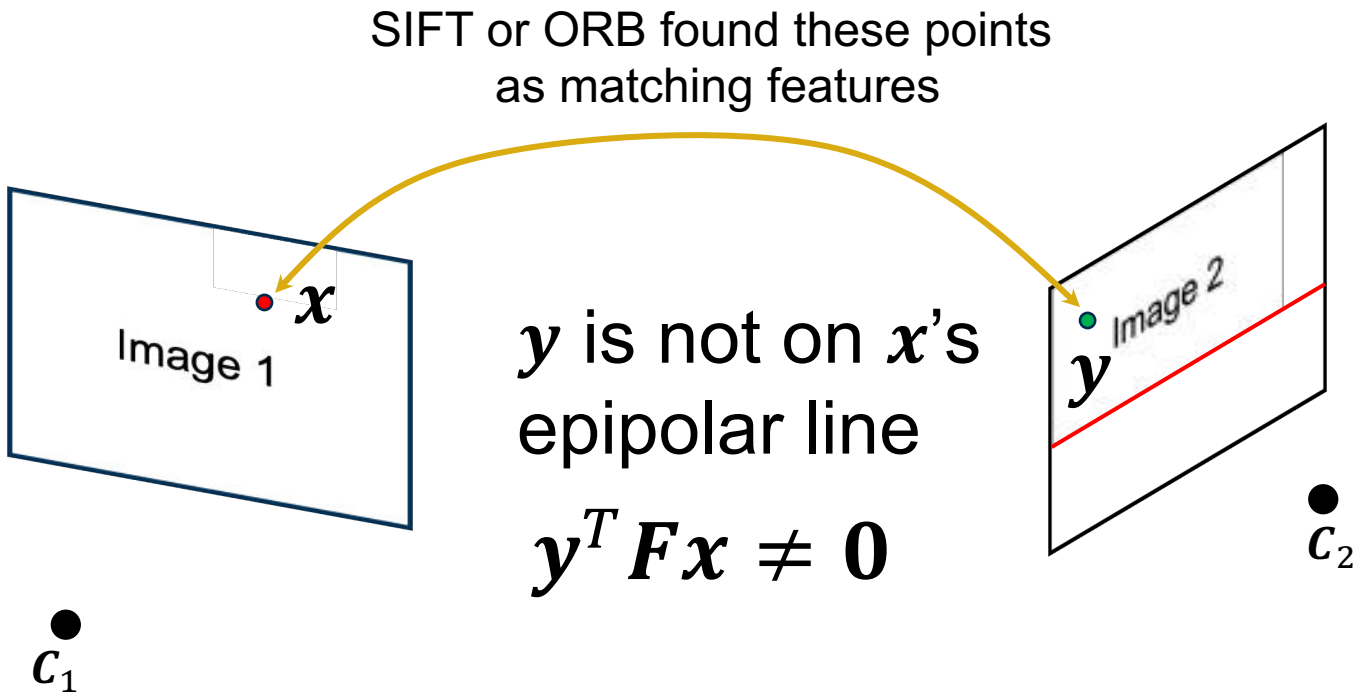
CRICOS No.00213J

Check for correctness of matches



CRICOS No.00213J

Check for correctness of matches

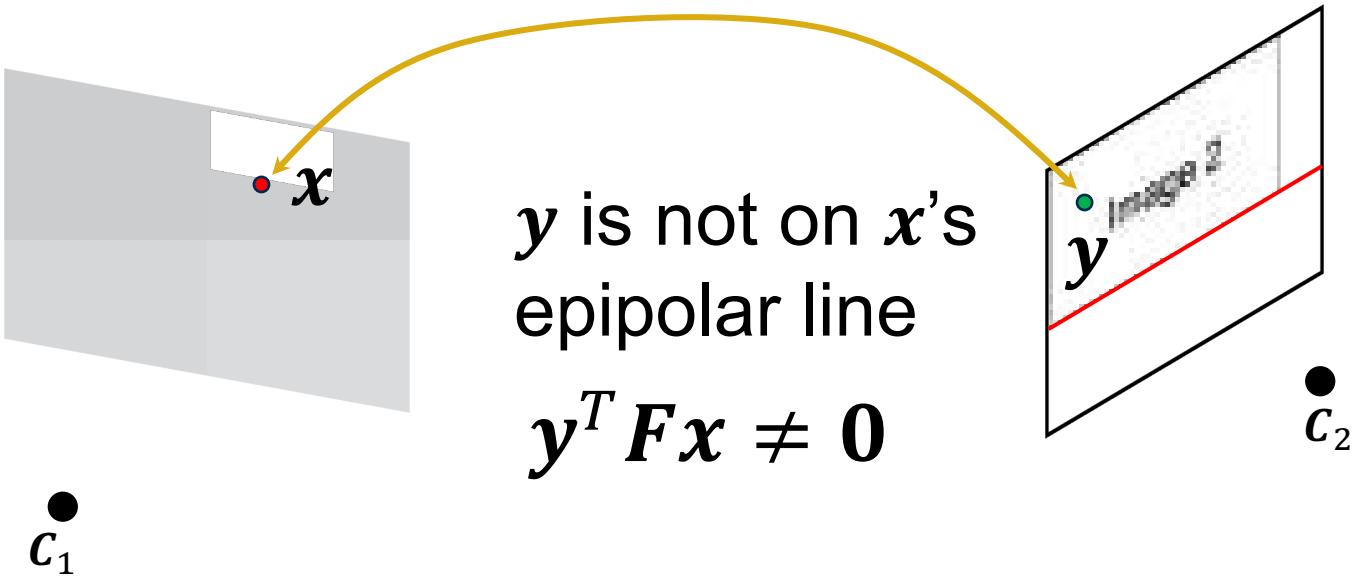


Given F, we can check if x and y are corresponding image points, i.e. if they are correct matches.

CRICOS No.00213J

Check for correctness of matches

SIFT or ORB found these points as matching features



y is not on x's epipolar line

$$y^T F x \neq 0$$

Use the fundamental matrix equation as a simple test to confirm or refute a match hypothesis:

If $x'^T F x = 0$: correct match

If $x'^T F x \neq 0$: incorrect match

Given F, we can check if x and y are corresponding image points, i.e. if they are correct matches.

Check for correctness of matches

But we need correct matches to calculate $\boldsymbol{F}$ in the first place.

What to do?



Use the fundamental matrix equation as a simple test to confirm or refute a match hypothesis:

If $\boldsymbol{x'^T F x = 0}$: correct match

If $\boldsymbol{x'^T F x \neq 0}$: incorrect match

CRICOS No.00213J

Check for correctness of matches

But we need correct matches to calculate $\boldsymbol{F}$ in the first place.

What to do?



Use the fundamental matrix equation as a simple test to confirm or refute a match hypothesis:

If $\boldsymbol{x'^T F x = 0}$: correct match

If $\boldsymbol{x'^T F x \neq 0}$: incorrect match

Powerful Idea:

1. *Calculate $\boldsymbol{F}$ from a small subset of the proposed matches (need 8)*
2. *Check which of the proposed matches do not pass the test*
3. *Repeat until we find a $\boldsymbol{F}$ for which a sufficiently large set passes the test*

Random Sample Consensus (RANSAC, 1981)

RANSAC:

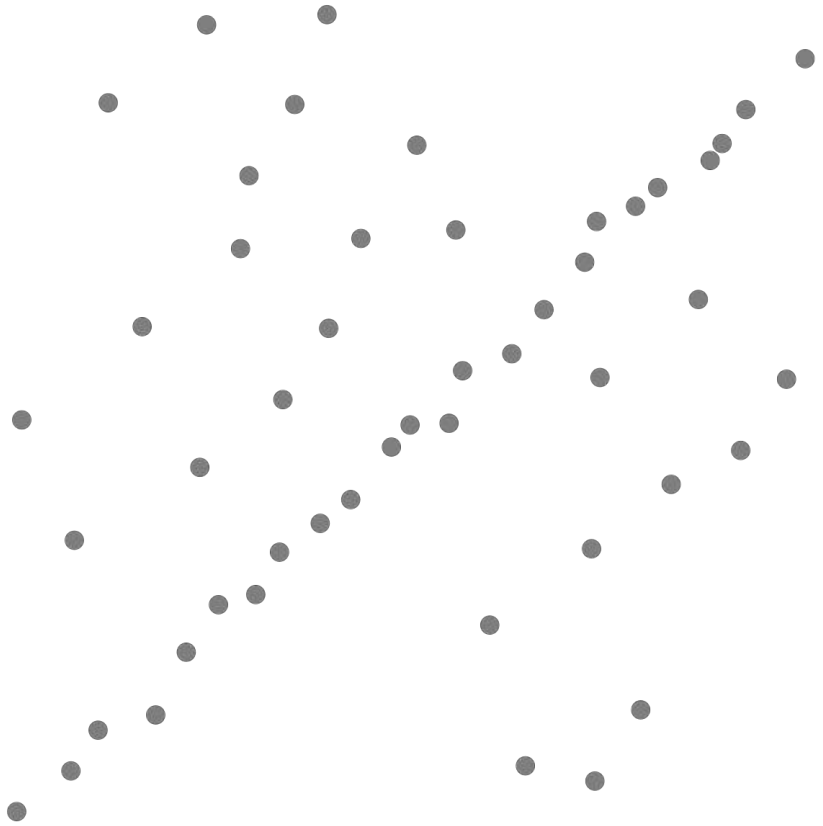
General setup: We want to estimate a model based on potentially noisy / erroneous data

1. *Calculate the model from a randomly selected minimal set (using as little of the data as possible)*
2. *Calculate the error between the model and the complete set of data*
3. *Data points with an error < threshold are "inliers" all data points with error > threshold are "outliers"*
4. *Repeat from 1 for k repetitions.*
5. *Best model is the one with the biggest inlier set.*

$$(\text{data} \rightarrow \text{sample} \rightarrow \text{fit model} \rightarrow \text{score} \rightarrow \text{repeat} \rightarrow \text{terminate})$$

CRICOS No.00213J

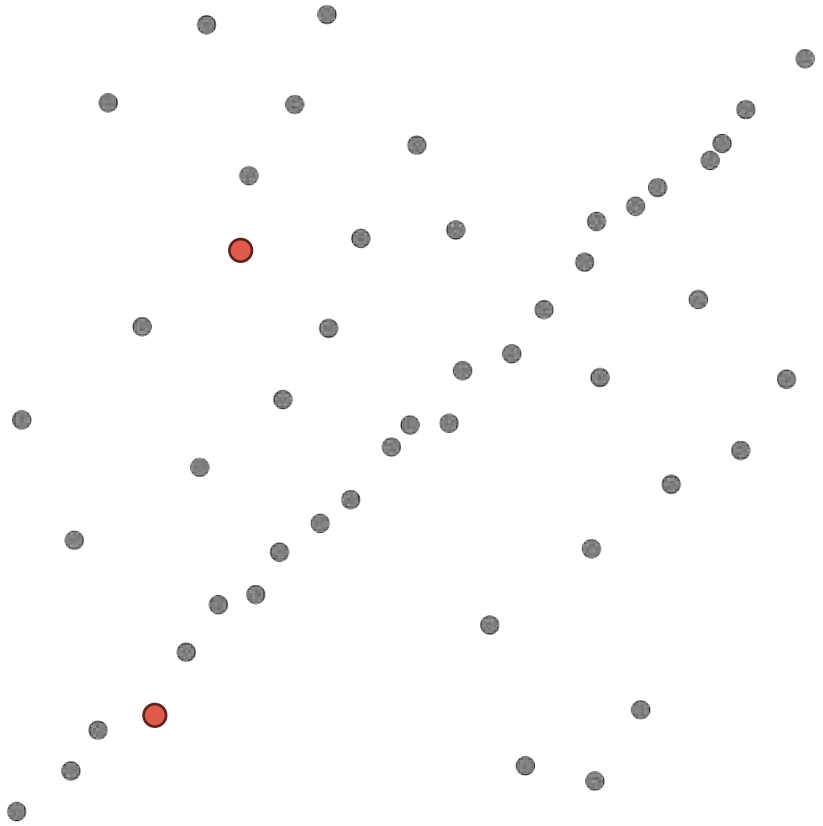
RANSAC Example: Line-Fitting



- Minimal set for a line in 2D?

CRICOS No.00213J

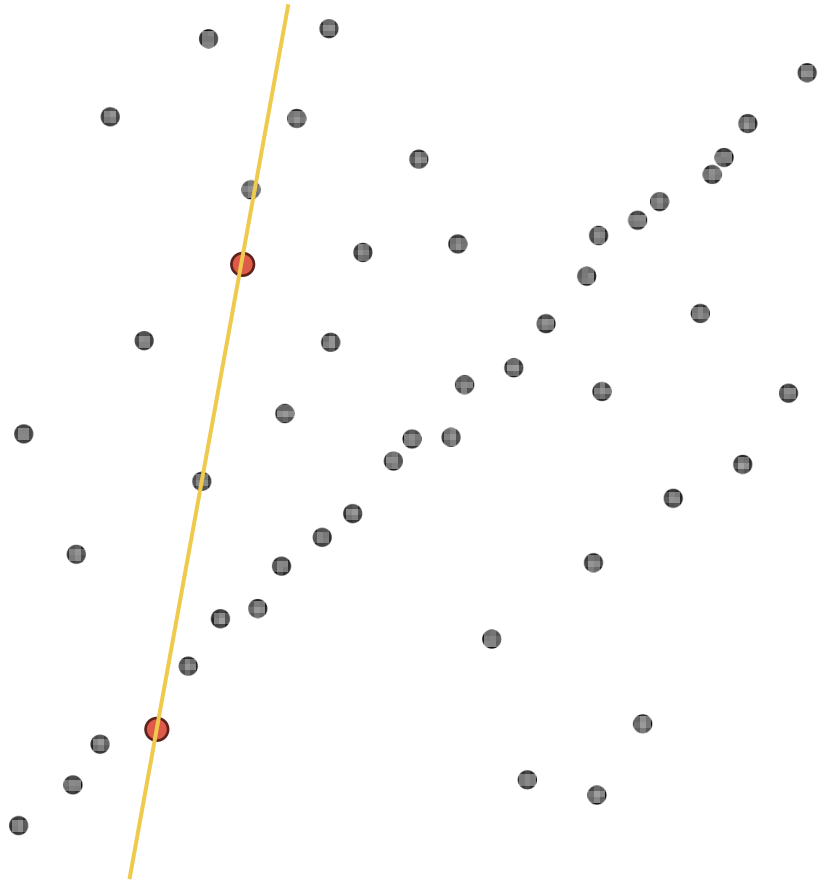
RANSAC Example: Line-Fitting



- Minimal set for a line in 2D?
- Select 2 points at random

CRICOS No.00213J

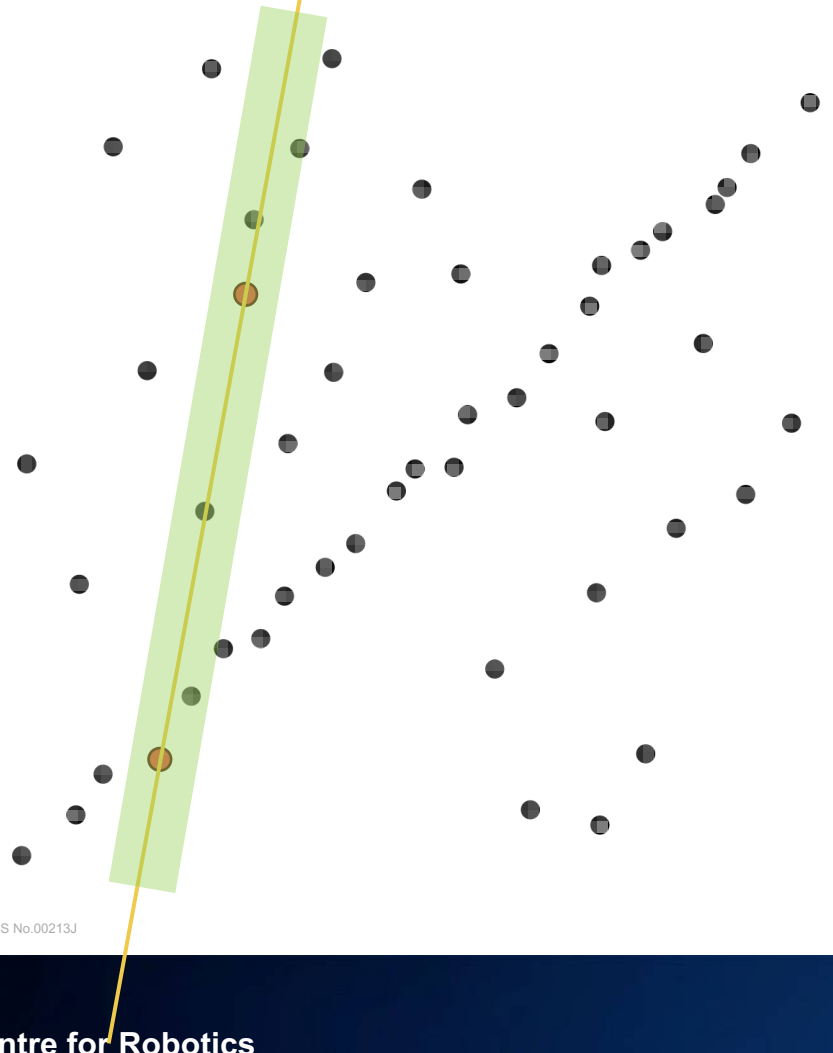
RANSAC Example: Line-Fitting



- Minimal set for a line in 2D?
- Select 2 points at random
- Fit a line

CRICOS No.00213J

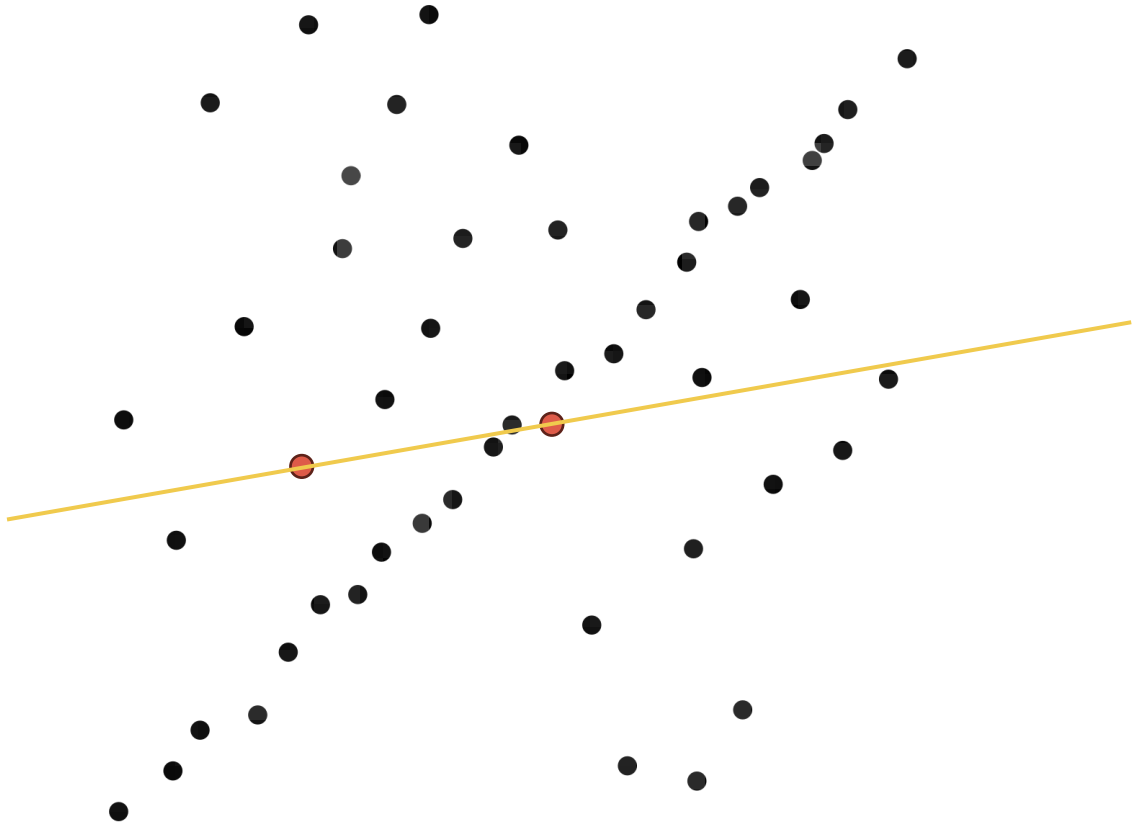
RANSAC Example: Line-Fitting



- Minimal set for a line in 2D?
- Select 2 points at random
- Fit a line
- Check how many points have acceptable (small) error.

CRICOS No.00213J

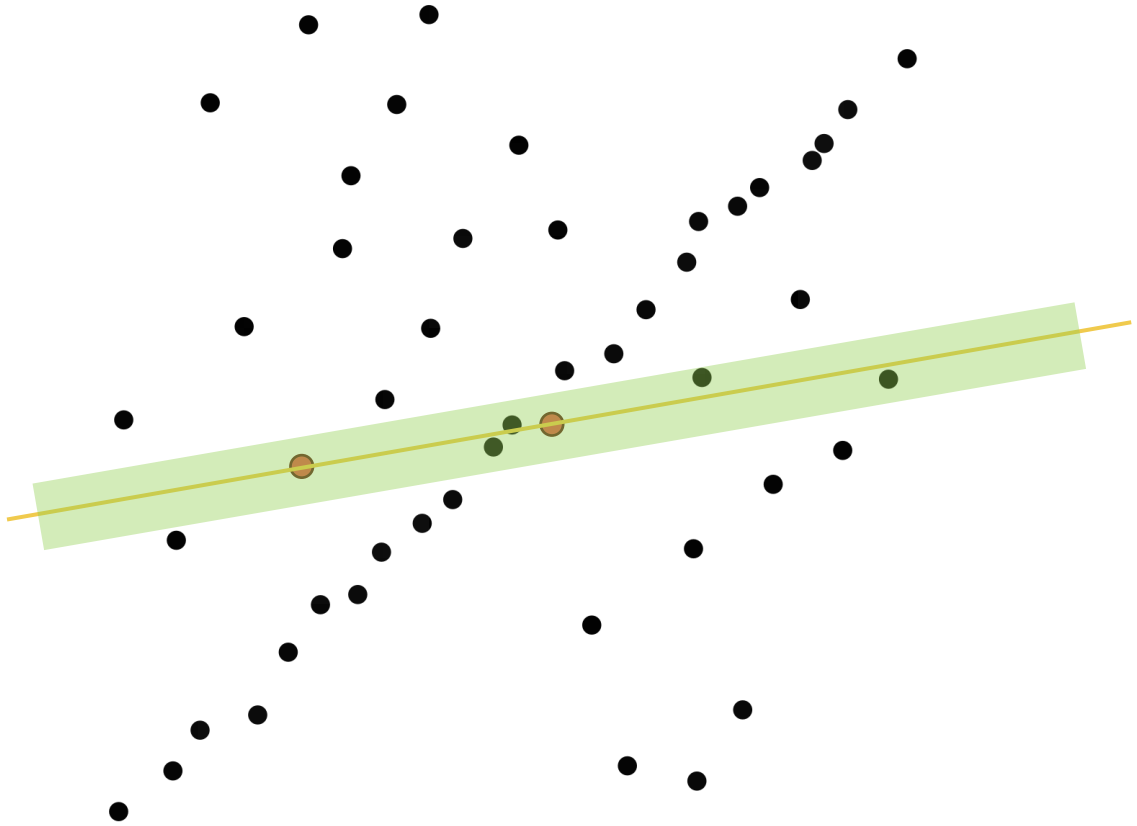
RANSAC Example: Line-Fitting



- Minimal set for a line in 2D?
- Select 2 points at random
- Fit a line
- Check how many points have acceptable (small) error.
- Repeat a certain number of times
- Remember the best solution found

CRICOS No.00213J

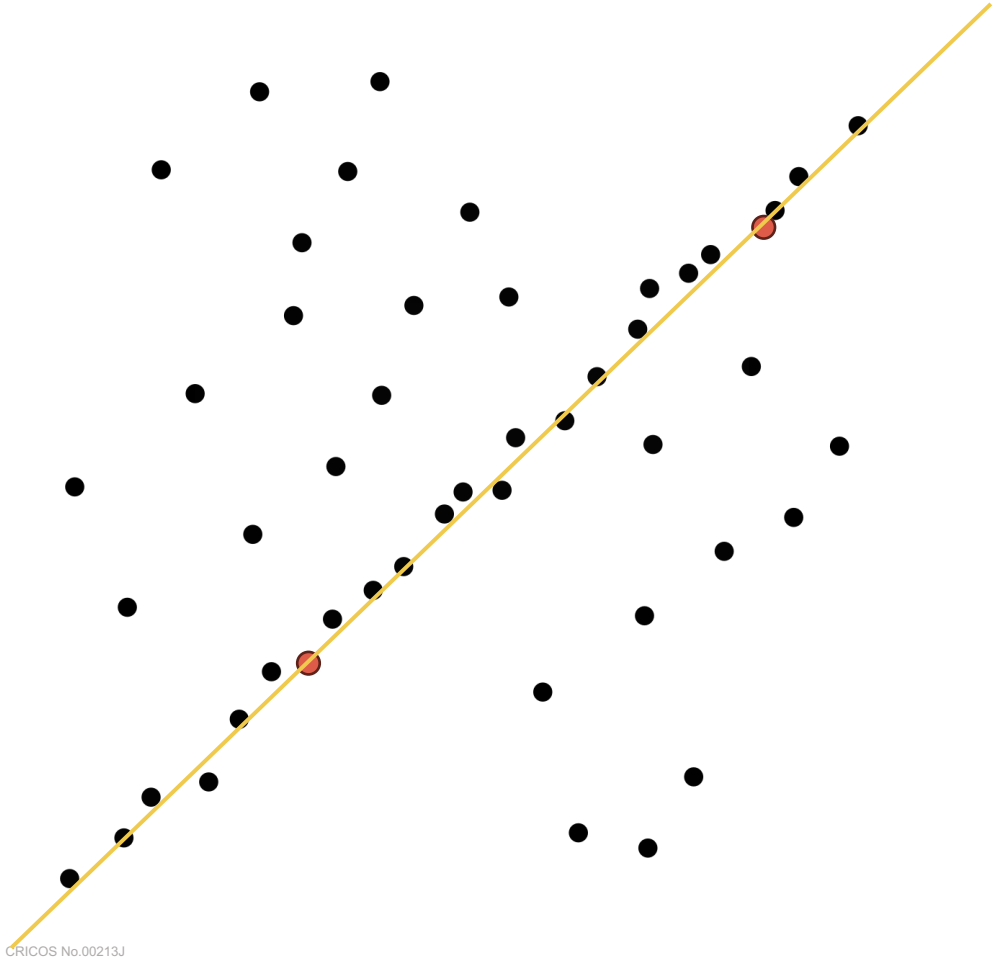
RANSAC Example: Line-Fitting



- Minimal set for a line in 2D?
- Select 2 points at random
- Fit a line
- Check how many points have acceptable (small) error.
- Repeat a certain number of times
- Remember the best solution found

CRICOS No.00213J

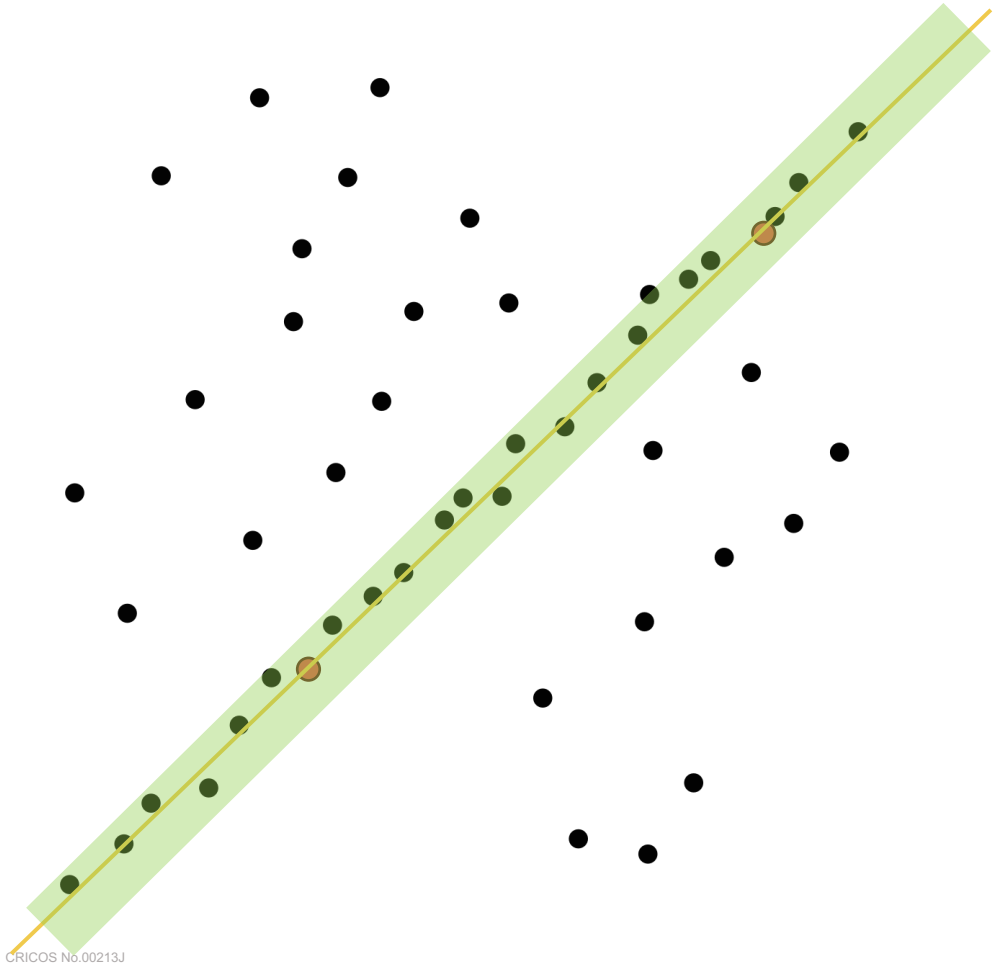
RANSAC Example: Line-Fitting



- Minimal set for a line in 2D?
- Select 2 points at random
- Fit a line
- Check how many points have acceptable (small) error.
- Repeat a certain number of times
- Remember the best solution found

CRICOS No.00213J

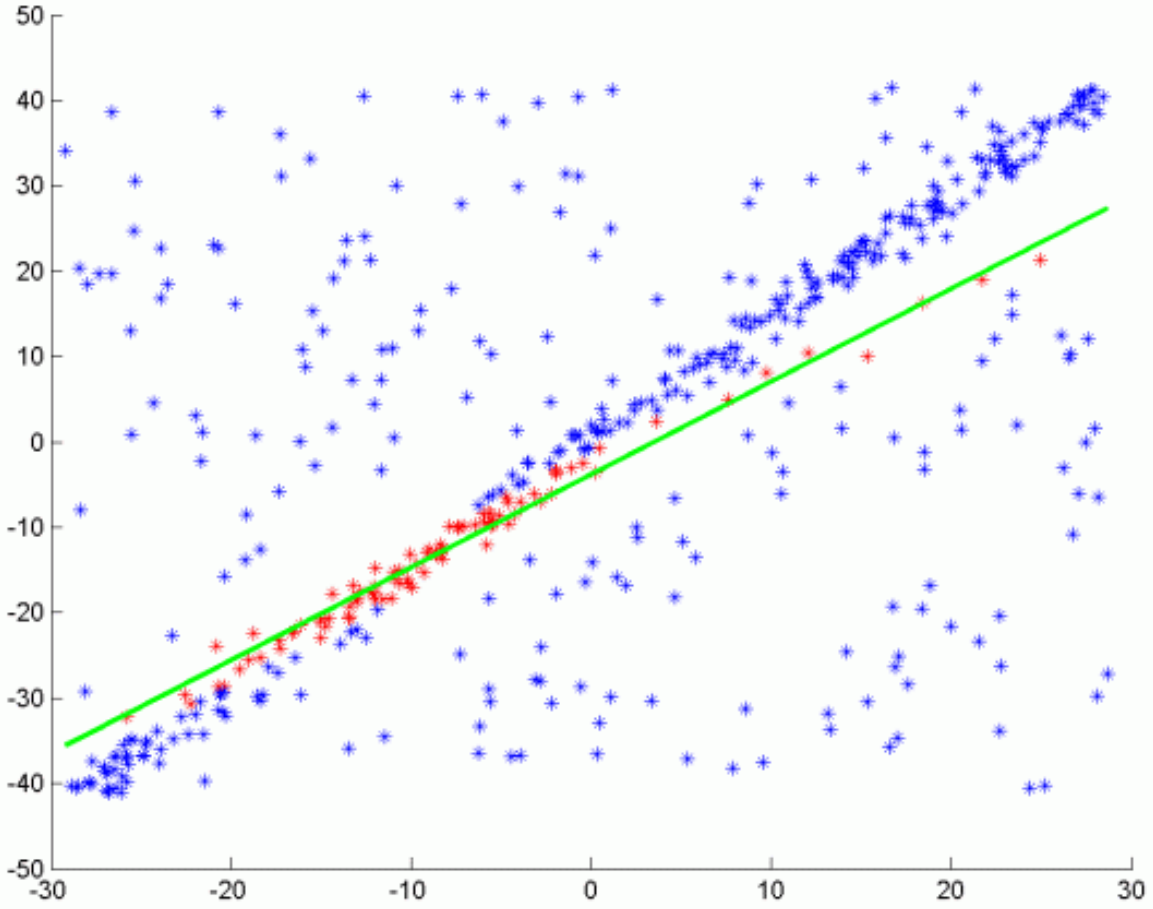
RANSAC Example: Line-Fitting



- Minimal set for a line in 2D?
- Select 2 points at random
- Fit a line
- Check how many points have acceptable (small) error.
- Repeat a certain number of times
- Remember the best solution found

CRICOS No.00213J

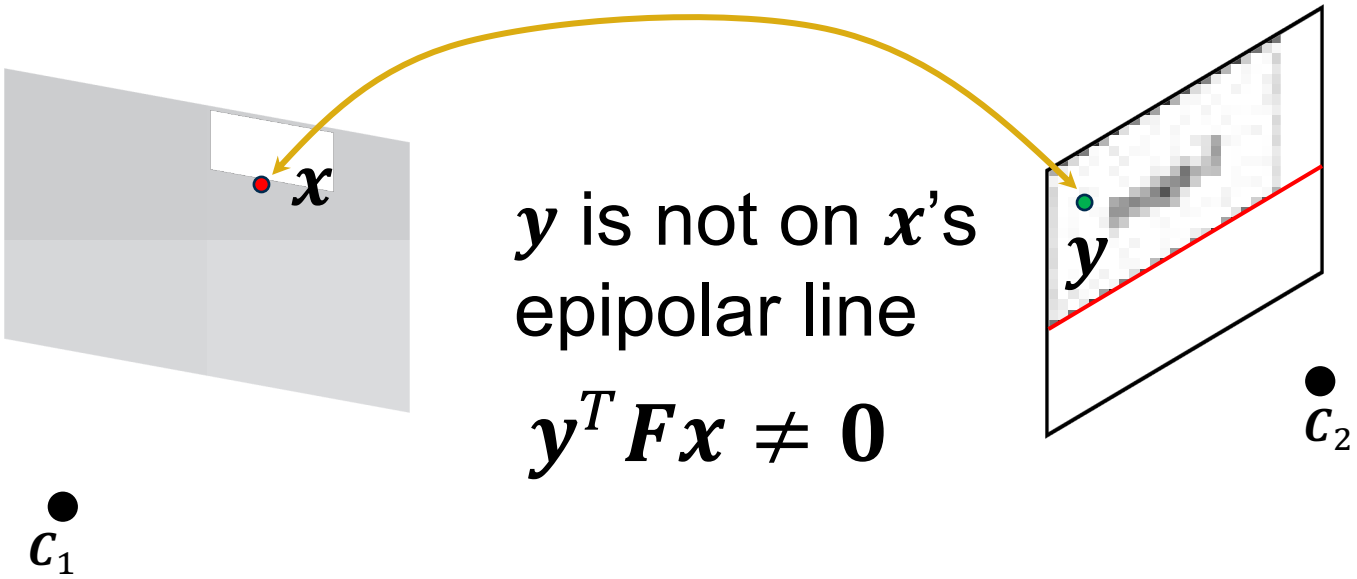
RANSAC Example: Line-Fitting



- Minimal set for a line in 2D?
- Select 2 points at random
- Fit a line
- Check how many points have acceptable (small) error.
- Repeat a certain number of times
- Remember the best solution found

Check for correctness of matches

SIFT or ORB found these points as matching features



y is not on x's epipolar line

$$y^T F x \neq 0$$

Use the fundamental matrix equation as a simple test to confirm or refute a match hypothesis:

If $x'^T F x = 0$: correct match

If $x'^T F x \neq 0$: incorrect match

Given F, we can check if x and y are corresponding image points, i.e. if they are correct matches.

CRICOS No.00213J

Fundamental Matrix with RANSAC to Identify False Feature Matches

- Calculate ***F*** from smallest possible number n of point matches
  - → choose 8 pairs ***x*** and ***x'*** at random
- Check all other pairs:
  - If they meet $\boldsymbol{x'^T F x = 0}$ then ***x*** and ***x'*** are inliers (correct match)
  - If not $\boldsymbol{x'^T F x = 0}$ then they are outliers (incorrect match)
- Repeat this k times, keep track of which ***F*** had most inliers

- After k repetitions:
  - Correct matches are the ones in the biggest inlier set

CRICOS No.00213J

RANSAC to Identify False Matches

- How many repetitions k?
  - Depends on expected ratio of w = inliers / all data
  - Depends on how many data points we need to calculate the model
    - (e.g. n=8 for Fundamental Matrix, n=2 for a 2D line, …)
  - Depends on p, the desired probability that there are no outliers in at least one of the minimal sets chosen in step 1

$$k = \frac{\log(1-p)}{\log(1-w^n)}$$

CRICOS No.00213J

RANSAC to Identify False Matches

- How many repetitions k?
  - Depends on expected ratio of w = inliers / all data
  - Depends on how many data points we need to calculate the model (n=8 for F)
  - Depends on p, the desired probability that there are no outliers in at least one of the minimal sets chosen in step 1

$$k = \frac{\log(1-p)}{\log(1-w^n)}$$

w = 90%; n = 8; p = 99.9% → k = 13
w = 75%; n = 8; p = 99.9% → k = 65

CRICOS No.00213J

RANSAC Improvements

(data → sample → fit model → score → repeat → terminate)

Limitation	Representative variant	Key idea and advantage
Uniform sampling wastes trials	**PROSAC**	Samples high-ranked observations first; faster when a useful confidence ranking exists
Good hypotheses are not refined	**LO-RANSAC**	Locally re-estimates promising models; improves accuracy and repeatability
Verification can be expensive	**Preemptive methods**	Reject poor hypotheses after examining only a few observations
Labels ignore spatial structure	**GC-RANSAC**	Encourages neighbouring observations to have consistent labels
One hard threshold is brittle	**MAGSAC++**	Scores across plausible noise scales; reduces sensitivity to threshold selection
Combine above improvements	**USAC**	**Provides a modular framework combining sampling, scoring, verification and refinement**

Raguram, R., Chum, O., Pollefeys, M., Matas, J., & Frahm, J. M. (2012).
USAC: A universal framework for random sample consensus. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *35*(8), 2022-2038.

CRICOS No.00213J

What else can we do with the Fundamental Matrix?

If we know the camera intrinsics, we can obtain information about the relative pose (rotation and translation) between the cameras.

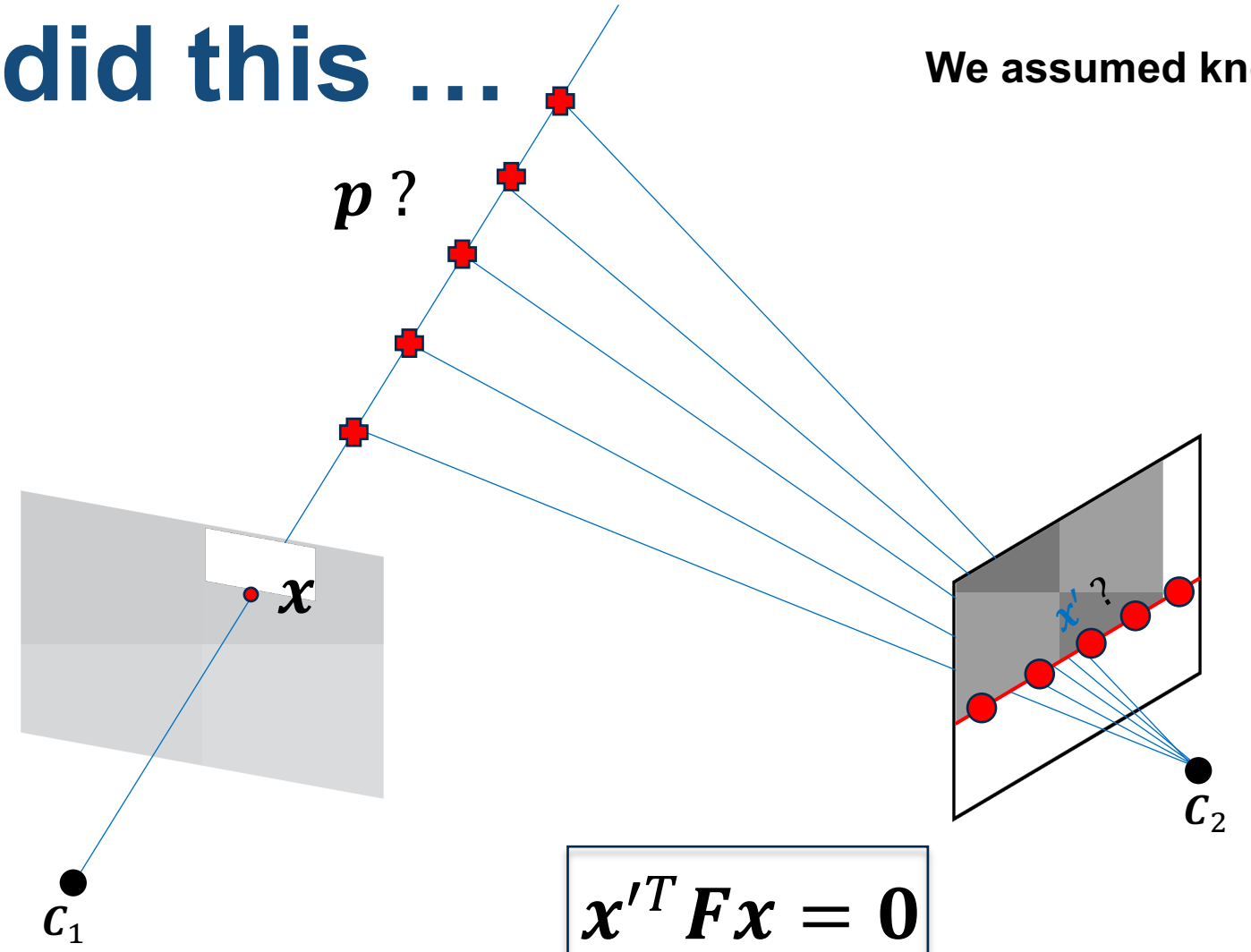
CRICOS No.00213J



The Essential Matrix and Motion from 2D-2D Matches

We did this …

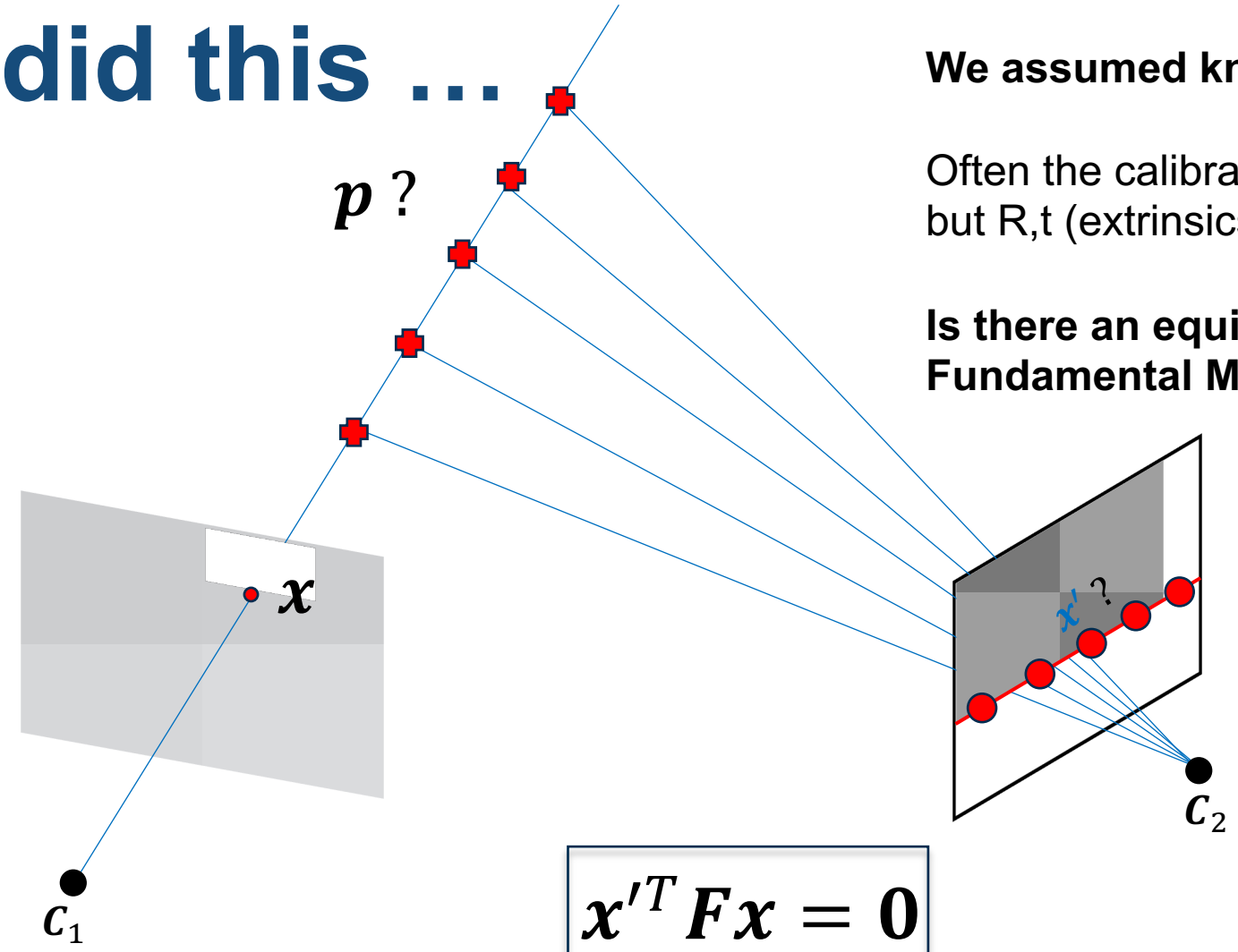
We assumed know nothing about the cameras.



$$x'^T F x = 0$$

This is true for **any x** and corresponding **x'**!

We did this …



We assumed know nothing about the cameras.

Often the calibration K (intrinsics) is known,
but R,t (extrinsics) are not.

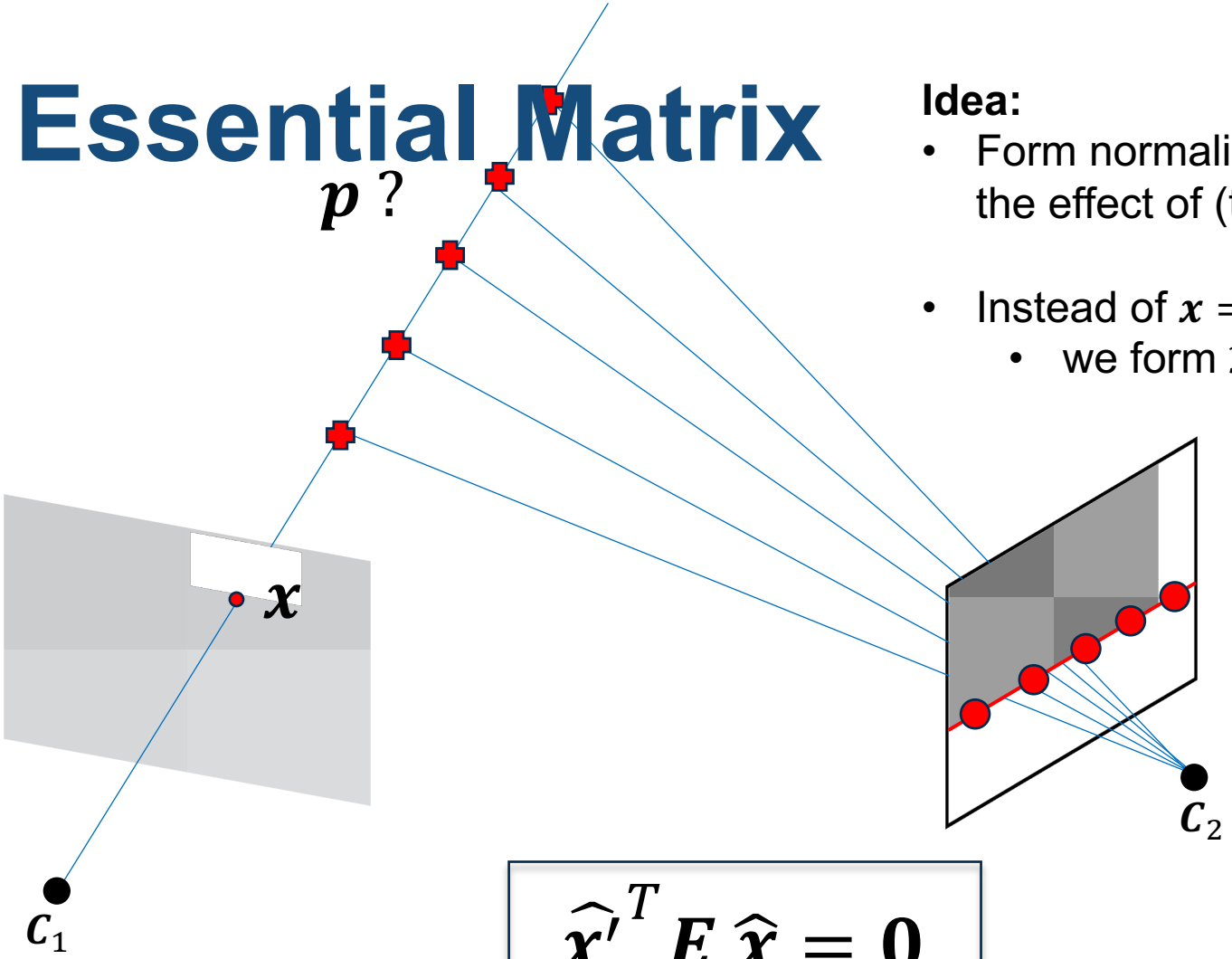
Is there an equivalent concept for the Fundamental Matrix, when K and K' are known?

$$x'^T F x = 0$$

This is true for **any x** and corresponding **x'**!

CRICOS No.00213J

The Essential Matrix



Idea:

- Form normalised image coordinates, by removing the effect of (the known) K.

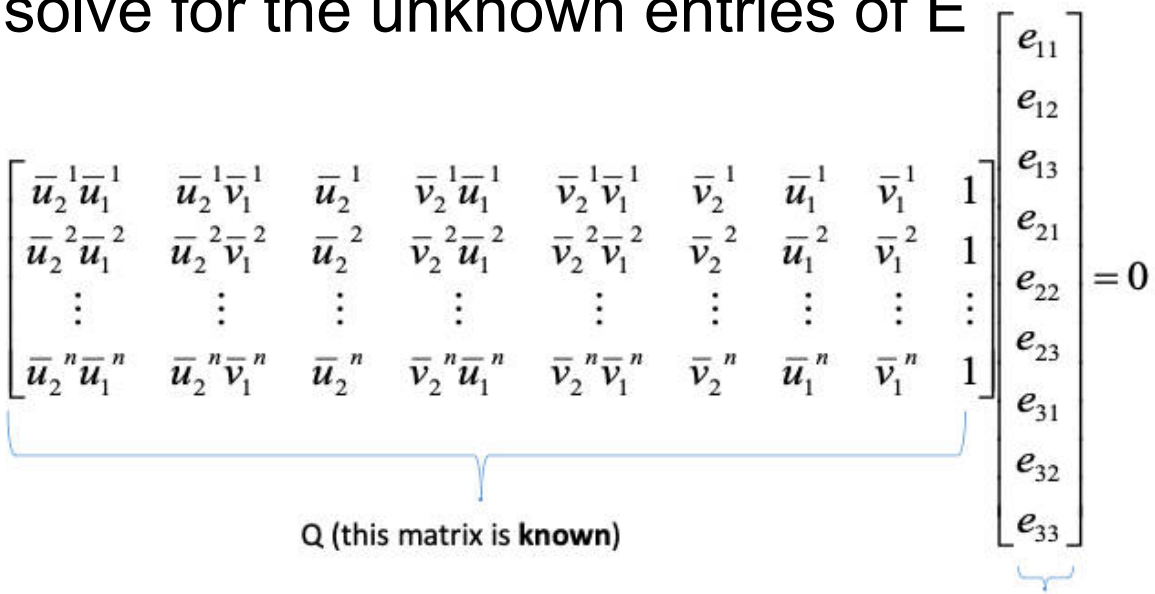
- Instead of $\boldsymbol{x} = \boldsymbol{K}[\boldsymbol{R}|\boldsymbol{t}] \cdot \boldsymbol{p}$ …
  - we form $\hat{\boldsymbol{x}} = \boldsymbol{K}^{-1} \cdot \boldsymbol{x}$
    $= \boldsymbol{K}^{-1}\boldsymbol{K}[\boldsymbol{R}|\boldsymbol{t}] \cdot \boldsymbol{p} = [\boldsymbol{R}|\boldsymbol{t}] \cdot \boldsymbol{p}$

$$\hat{\boldsymbol{x}}'^{T} \boldsymbol{E}\, \hat{\boldsymbol{x}} = \boldsymbol{0}$$

This is true for **any x** and corresponding **x'**!

Computing the Essential Matrix

- Can be estimated from at least 5 point correspondences
- Use OpenCV's function `findEssentialMat()`
  - Similar to the Fundamental Matrix algorithm: form a system of equations and solve for the unknown entries of E




```
[U,S,V] = svd(Q);
Ev = V(:,9);
E = reshape(Ev,3,3)';
```

CRICOS No.00213J

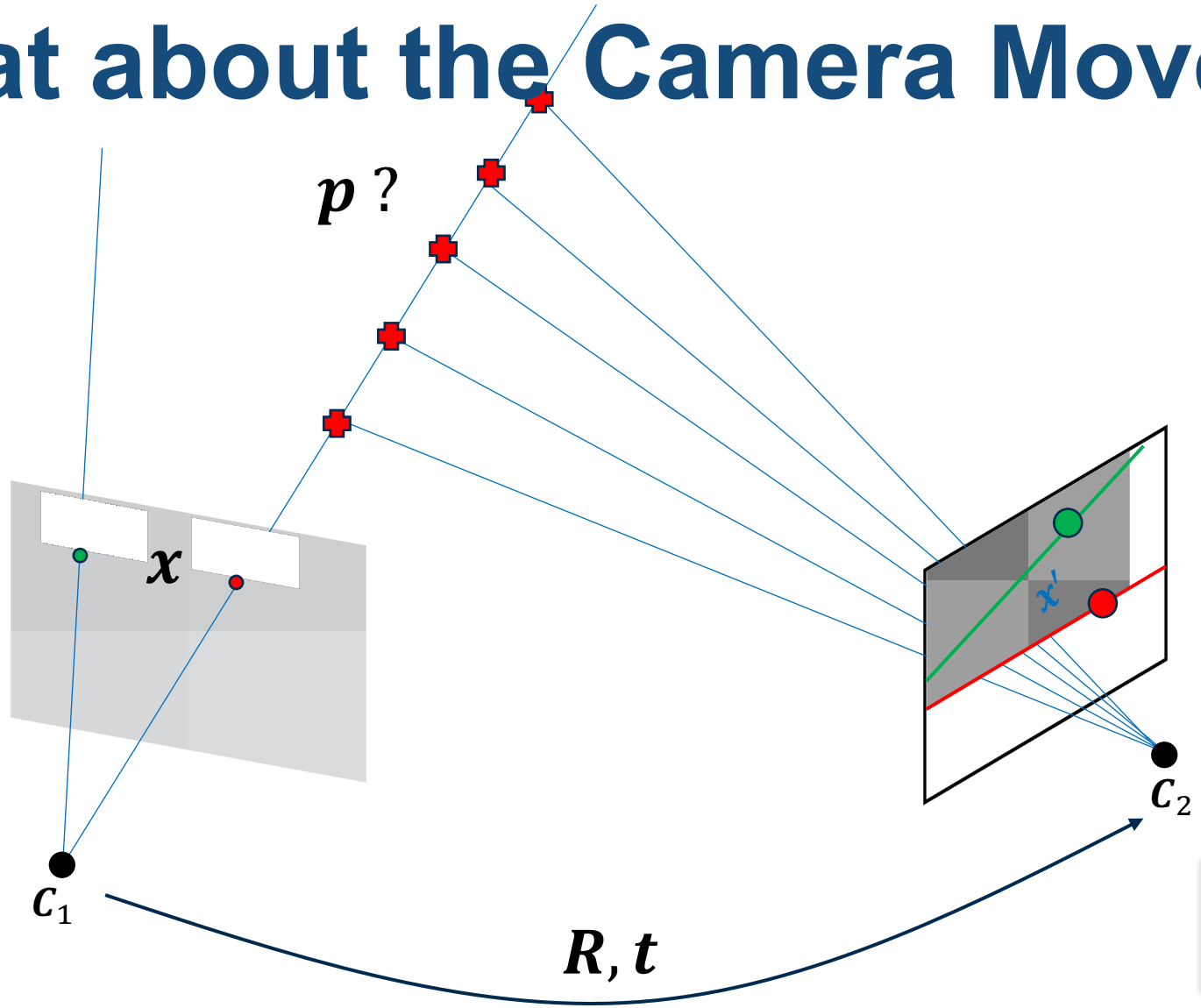
Illustration of equations courtesy of Davide Scaramuzza

Computing the Essential Matrix

- Can be estimated from at least 5 point correspondences
- Use OpenCV's function `findEssentialMat()`
  - Similar to the Fundamental Matrix algorithm: form a system of equations and solve for the unknown entries of E
- We can also compute E from the Fundamental matrix and the two intrinsic matrices K, K': $\boldsymbol{E} = \boldsymbol{K'}^{T}\boldsymbol{F}\boldsymbol{K}$

CRICOS No.00213J

What about the Camera Movement?



It seems that E must encode some information about R and t?

After all, if the second camera moves, the epipolar lines will move and E must change.

$$\hat{x}'^{T} E \hat{x} = 0$$

This is true for **any x** and corresponding **x'**!

CRICOS No.00213J

Recovering R and t from E

We can always assume this is the case!

Consider a pair of normalized cameras with projection matrices $[\boldsymbol{I}|\boldsymbol{0}] \cdot \boldsymbol{p}$ and $[\boldsymbol{R}|\boldsymbol{t}] \cdot \boldsymbol{p}$.

E has a very special form. It contains the relative motion between both cameras.

$$\boldsymbol{E} = [\boldsymbol{t}]_{\times}\boldsymbol{R}$$

Here $[\boldsymbol{t}]_{\times}$ is the skew-symmetric matrix of t: $[\boldsymbol{t}]_{\times} = \begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix}$

CRICOS No.00213J

Recovering R and t from E

We can always assume this is the case!

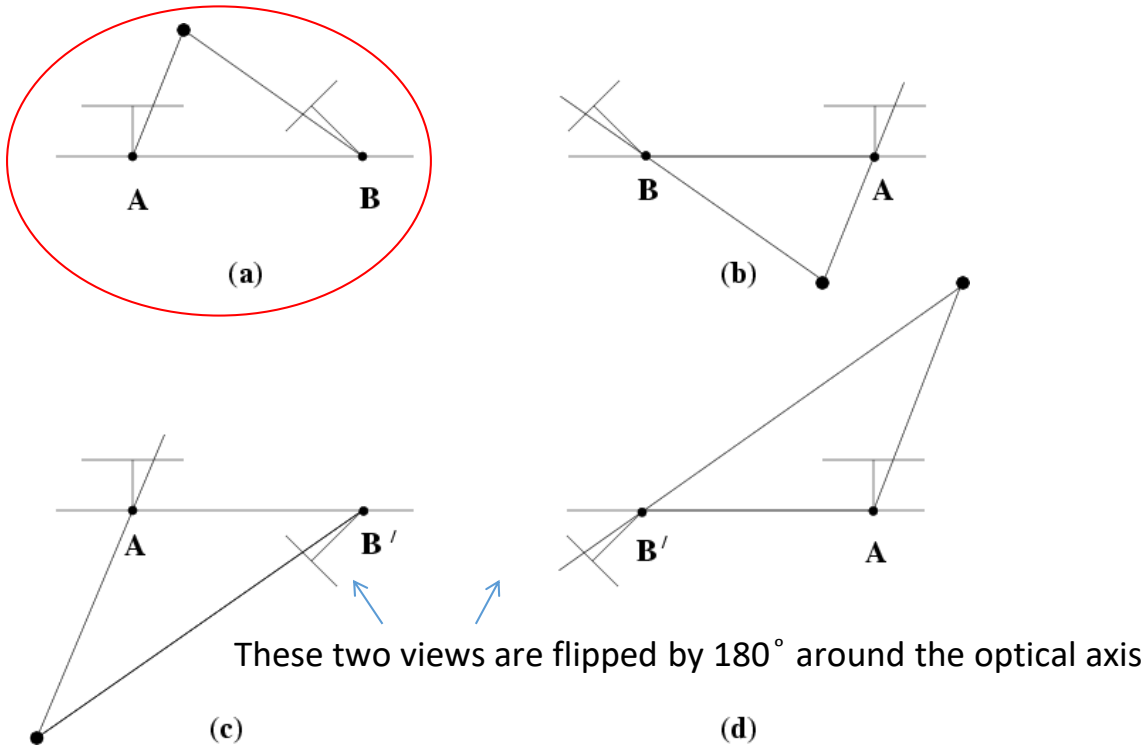
Consider a pair of normalized cameras with projection matrices $[\boldsymbol{I}|\boldsymbol{0}] \cdot \boldsymbol{p}$ and $[\boldsymbol{R}|\boldsymbol{t}] \cdot \boldsymbol{p}$.

E has a very special form. It contains the relative motion between both cameras.

$$\boldsymbol{E} = [\boldsymbol{t}]_{\times}\boldsymbol{R}$$

Use OpenCV's `decomposeEssentialMat()` to recover **t** and **R** from **E**.
But there are 4 solutions …

Recovering R and t from E



CRICOS No.00213J Illustration courtesy of Davide Scaramuzza

Small inconvenience:

- There are 2 solutions for $\boldsymbol{R}$ and two solutions for $\boldsymbol{t}$!
  - $\boldsymbol{t}$ and $-\boldsymbol{t}$
  - $\boldsymbol{R_1}$and $\boldsymbol{R_2}$are twisted by 180deg around the baseline

- So in total there are 4 solutions:
  - $[\boldsymbol{R_1}, \boldsymbol{t}]$, $[\boldsymbol{R_1}, -\boldsymbol{t}]$, $[\boldsymbol{R_2}, \boldsymbol{t}]$, $[\boldsymbol{R_2}, -\boldsymbol{t}]$

- **Solution:**
  - match points between both images,
  - triangulate their 3D position.
  - In only one of the 4 solutions are points in front of both cameras.

- Use OpenCV's `trianglulatePoints()` to check which is the correct R and t.

For Convenience

OpenCV's recoverPose() does it all in one go:

- `R,t = recoverPose(E, points1, points2)`

CRICOS No.00213J

Hold on!

But hold on!

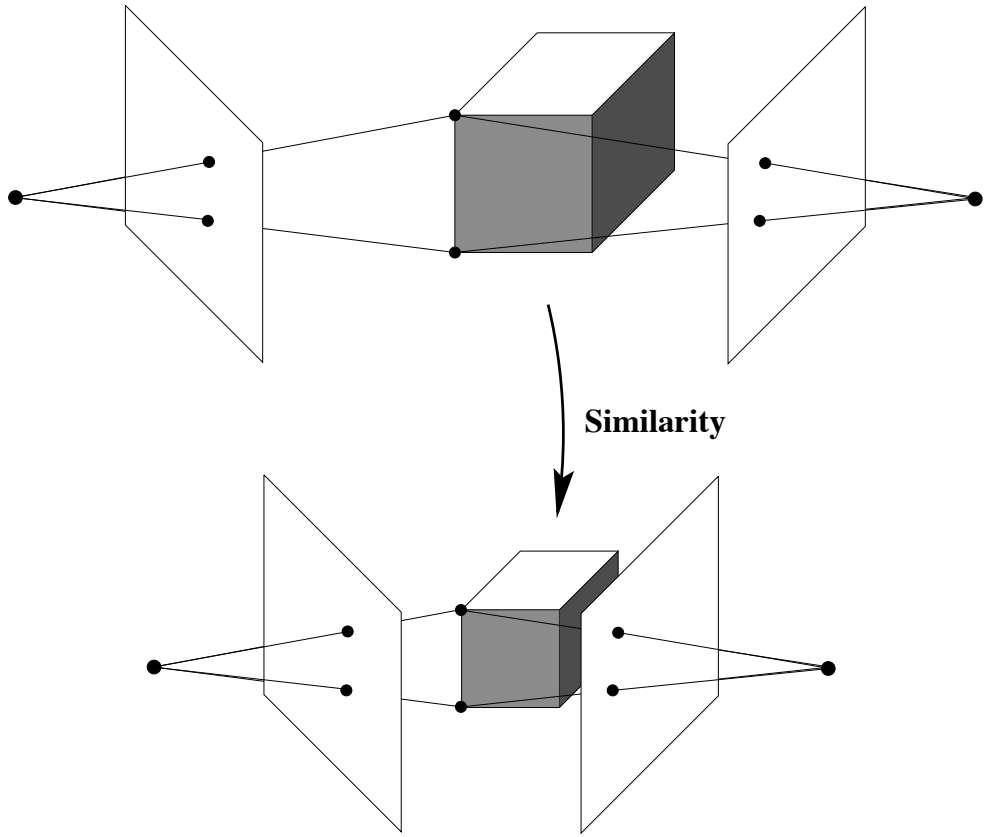
- We got R and t from E?
- So now we can treat both cameras as a stereo system!
- But that means we can get true depth information from the scene just by taking a camera and moving it?

Hold on!

But hold on!

- We got R and t from E?
- So now we can treat both cameras as a stereo system!
- But that means we can get true depth information from the scene just by taking a camera and moving it?

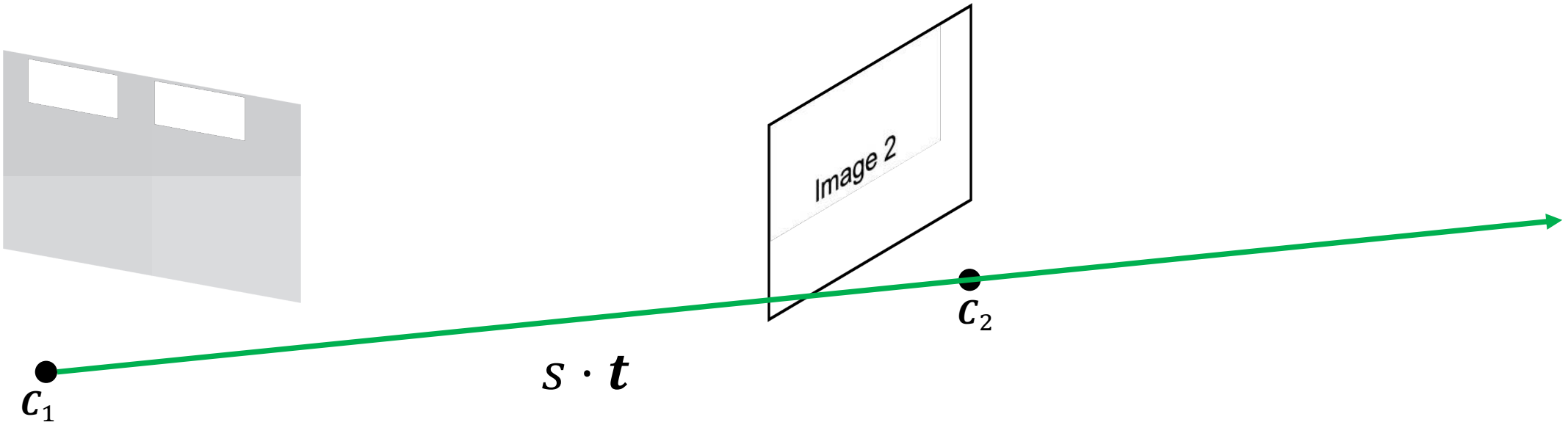
- **That should not be allowed!**



Multiple View Geometry in Computer Vision
Richard Hartley and Andrew Zisserman,
Cambridge University Press, Second Edition, 2004.

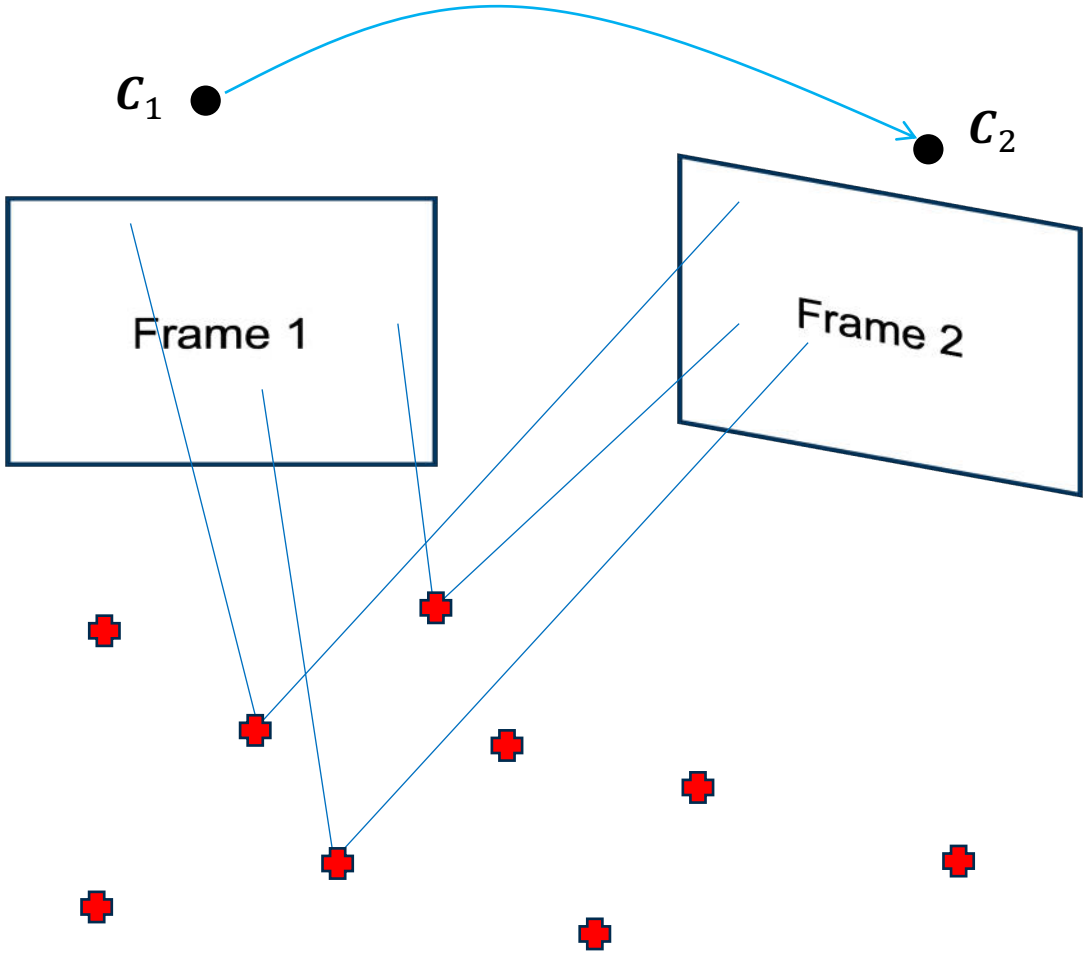
An Important Limitation

- **t** can only be recovered up-to-scale! Direction yes, but not distance
- **R** will be correct



We know the second camera centre C_2 is somewhere on the vector $s \cdot t$, but we do not know where.
The unknown scale factor s cannot be recovered without additional information.

How 2D-2D Methods are used

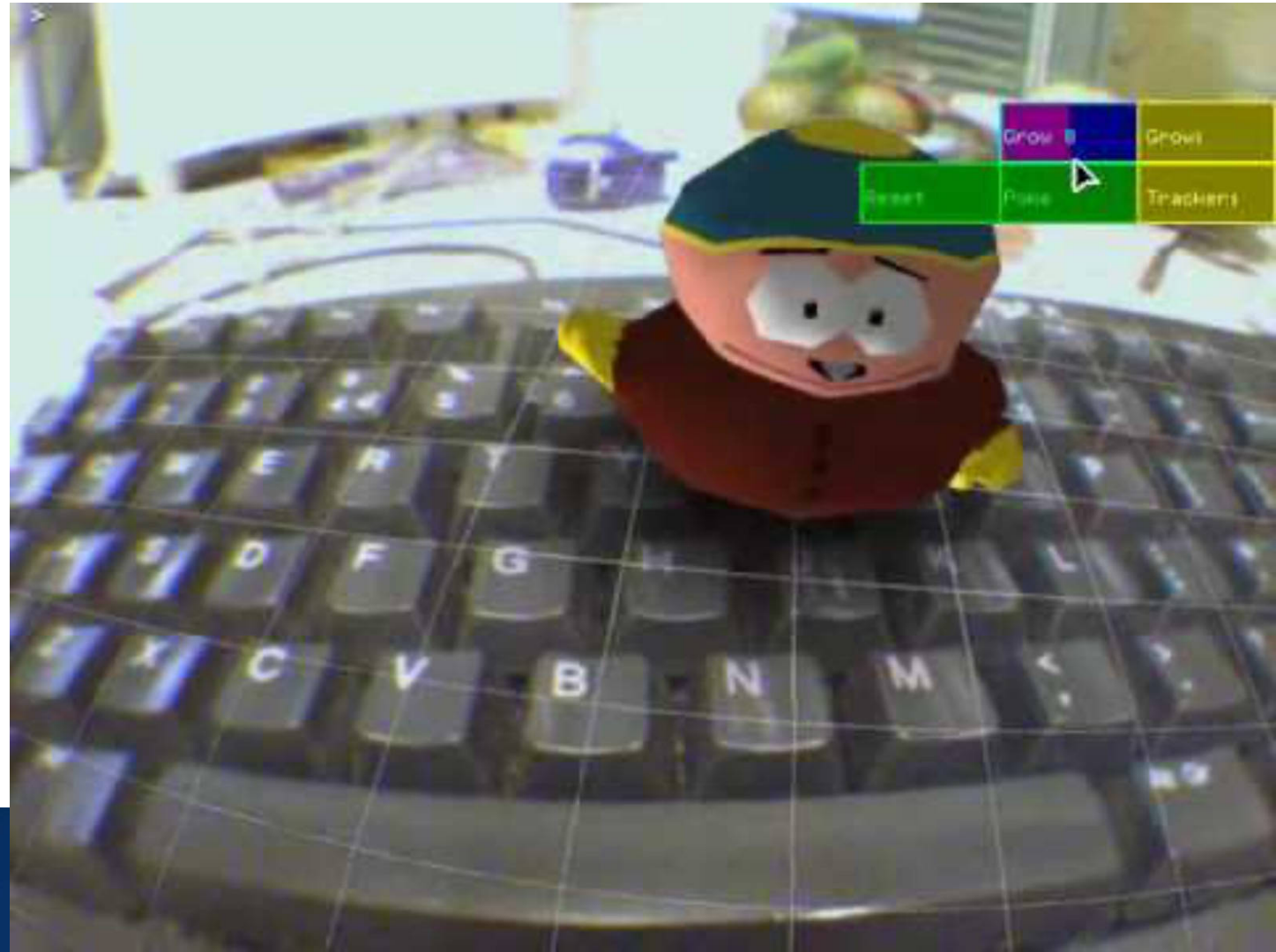


- Used to initialise monocular visual odometry
  - initialise a map by estimating R and t between the first two cameras
  - triangulate 3D points → map
  - accept that scale of the map and motion will be arbitrary
  - Then switch to 3D-2D method like PnP (next week)

CRICOS No.00213J

Case Study: PTAM

- For some applications, correct metric scale is actually not important.
- Augmented Reality (AR) is a good example.

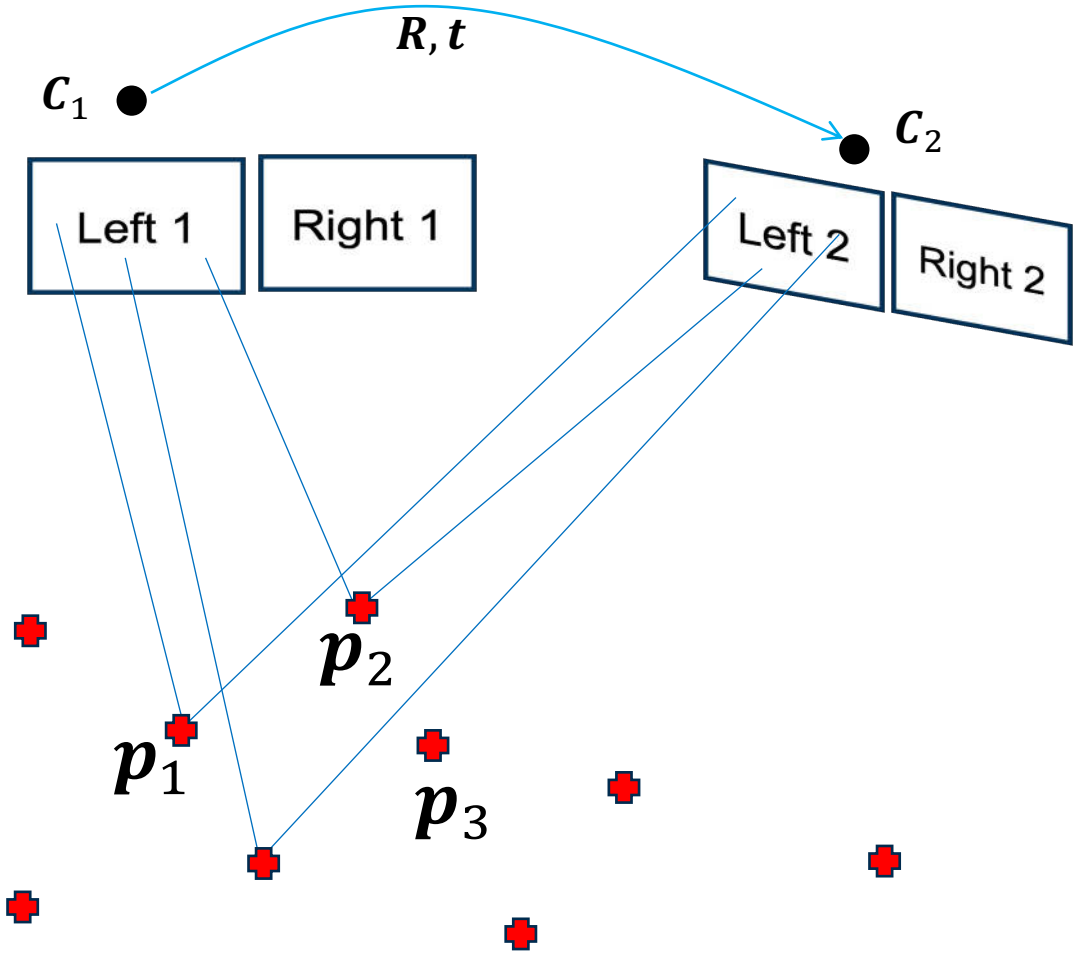


Case Study: Monocular Visual Odometry in the Oculus VR Headset



CRICOS No.00213J

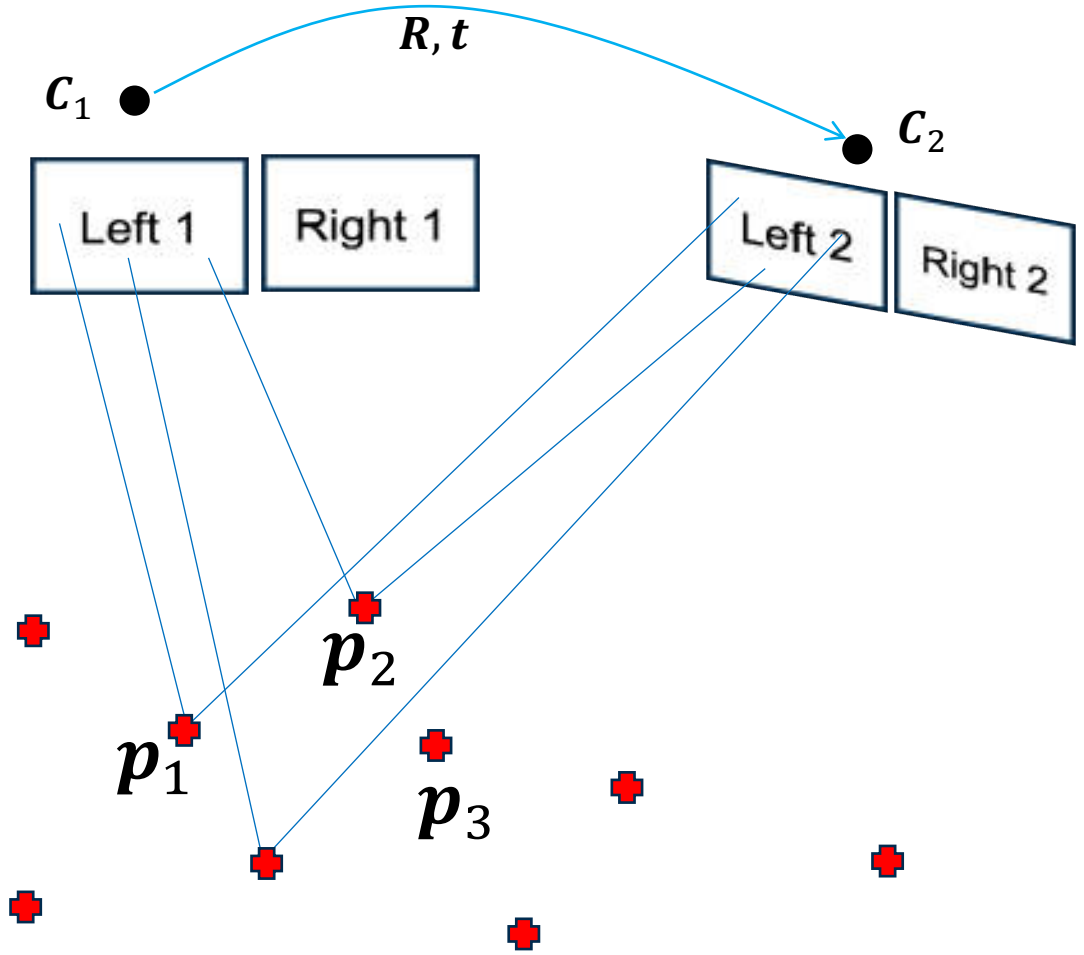
How 2D-2D Methods are used



- If we have access to the true depth of some points, we can recover the missing scale factor
  - E.g. with a stereo camera or depth camera
  - Hint: if we do have depth, we are generally better with 3D-2D methods (PnP) instead of 2D-2D

CRICOS No.00213J

Recover Scale from Known 3D Points

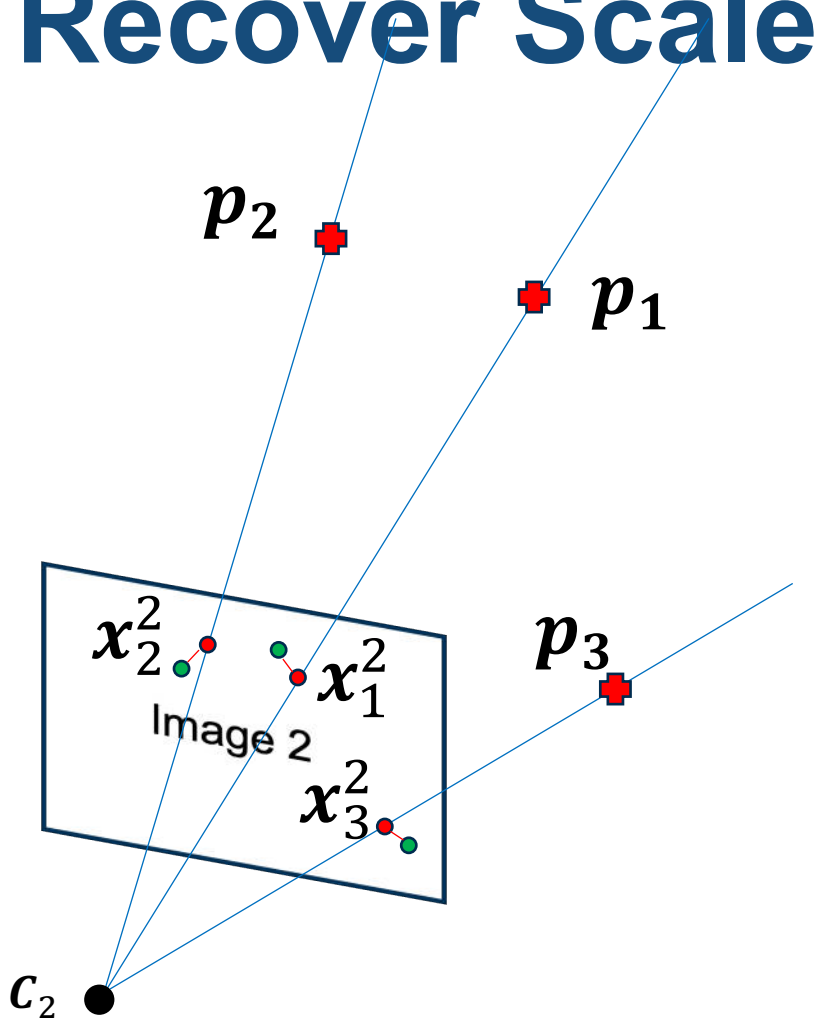


- Idea: We know the 3D coordinates p_i for some image points $\mathbf{x}_i^k$.
- We also know where that 3D point should project into the second camera, if we had the correct scale s:

$$\mathbf{x}_i^{k+1} = \boldsymbol{K}(\boldsymbol{R} \cdot \boldsymbol{p}_i + s \cdot \boldsymbol{t})$$

CRICOS No.00213J

Recover Scale from Known 3D Points



- Idea: We know the 3D coordinates $\boldsymbol{p_i}$ for some image points $\mathbf{x}_i^k$.
- We also know where that 3D point should project into the second camera, if we had the correct scale s:

$$\boldsymbol{K}(\boldsymbol{R} \cdot \boldsymbol{p}_i + s \cdot \boldsymbol{t})$$

- But if we have the wrong scale, we will observe the point at a different pixel.
- The difference is called the reprojection error
- Solution: Find the optimal scale that minimises this error

$$s* = \mathrm{argmin}_s \sum_i \left\| \boldsymbol{x}_i^{k+1} - \boldsymbol{K}(\boldsymbol{R} \cdot \boldsymbol{p}_i + s \cdot \boldsymbol{t}) \right\|^2$$

CRICOS No.00213J

Recover Scale from Known 3D Points

This is a nonlinear least-squares optimisation problem.

Problems of this structure are very common in robotic vision / computer vision.

See Practical on Friday on how to solve this using Scipy.

$$s* = \text{arg}min_s \sum_i \left\| \boldsymbol{x}_i^{k+1} - \boldsymbol{K}(\boldsymbol{R} \cdot \boldsymbol{p}_i + s \cdot \boldsymbol{t}) \right\|^2$$

CRICOS No.00213J

Different Problems, Different Solution

Problem structure	What we do	Typical tool
Exact, directly invertible equations	Solve for the unknowns	Rearrangement or linear solve
Noisy linear equations	Minimise linear residuals	QR-decomposition or SVD
Noisy nonlinear equations	Iteratively minimise residuals	Gauss–Newton or Levenberg-Marquardt
Outliers are present	Limit or reject their influence	Robust loss function or RANSAC

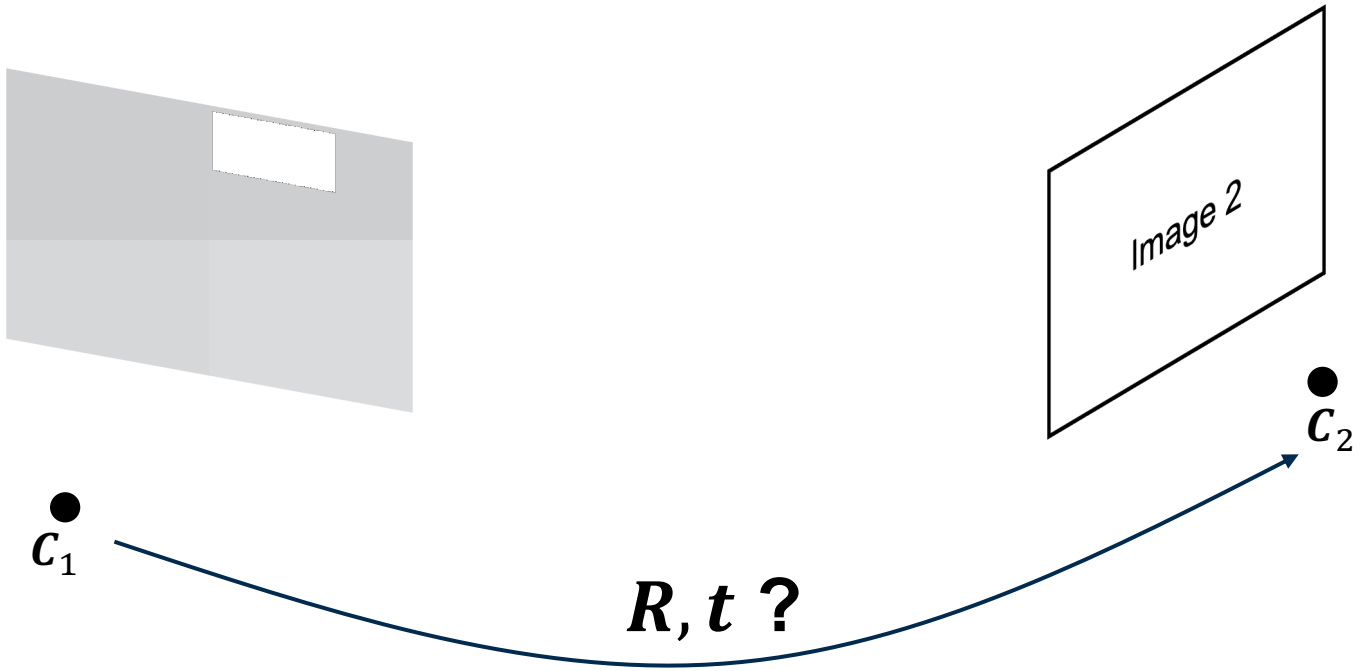
1. Is the measurement model exact or approximate?
2. Is it linear or nonlinear?
3. Are outliers expected?

CRICOS No.00213J

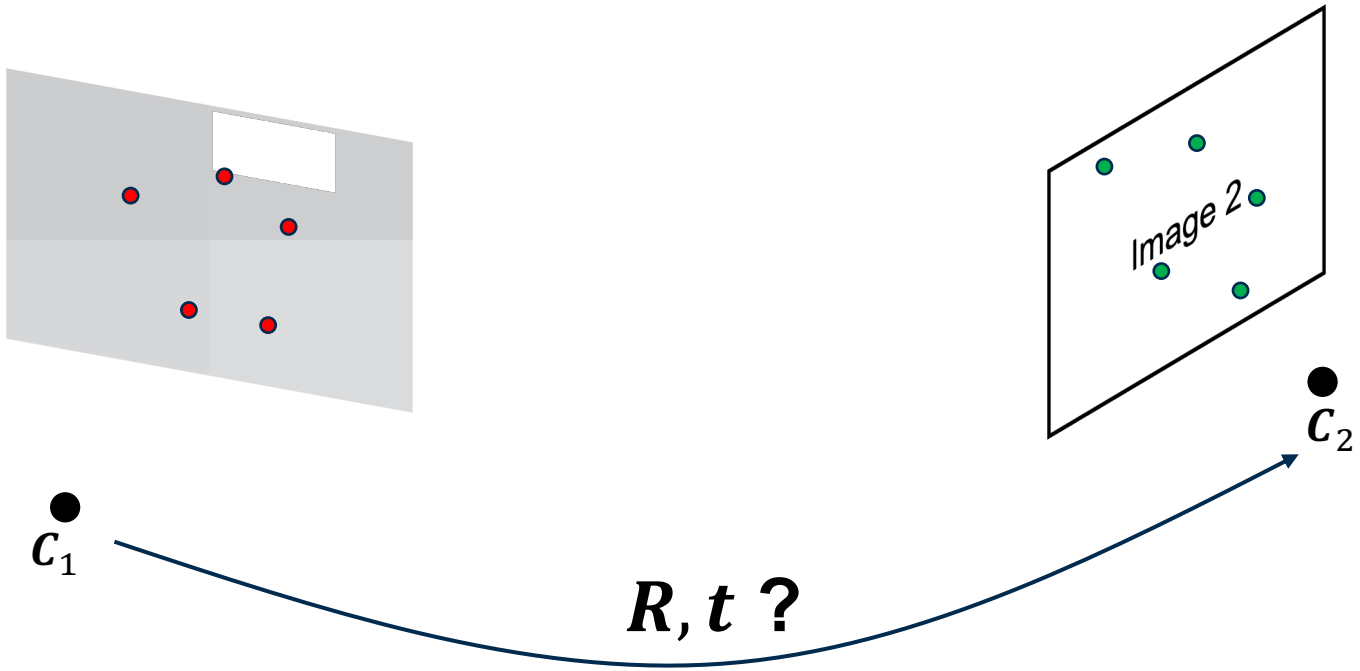


Recap

Summary: Motion from 2D-2D Matches



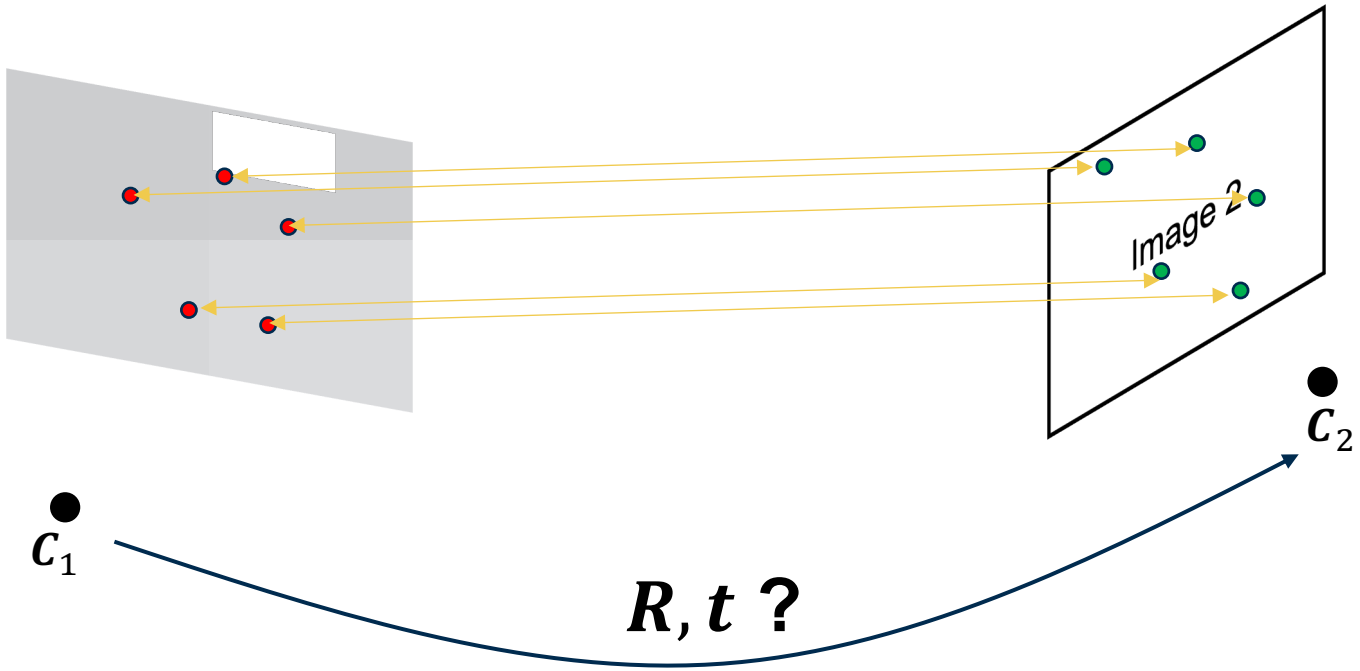
Summary: Motion from 2D-2D Matches



- Find interest points in both images.

CRICOS No.00213J

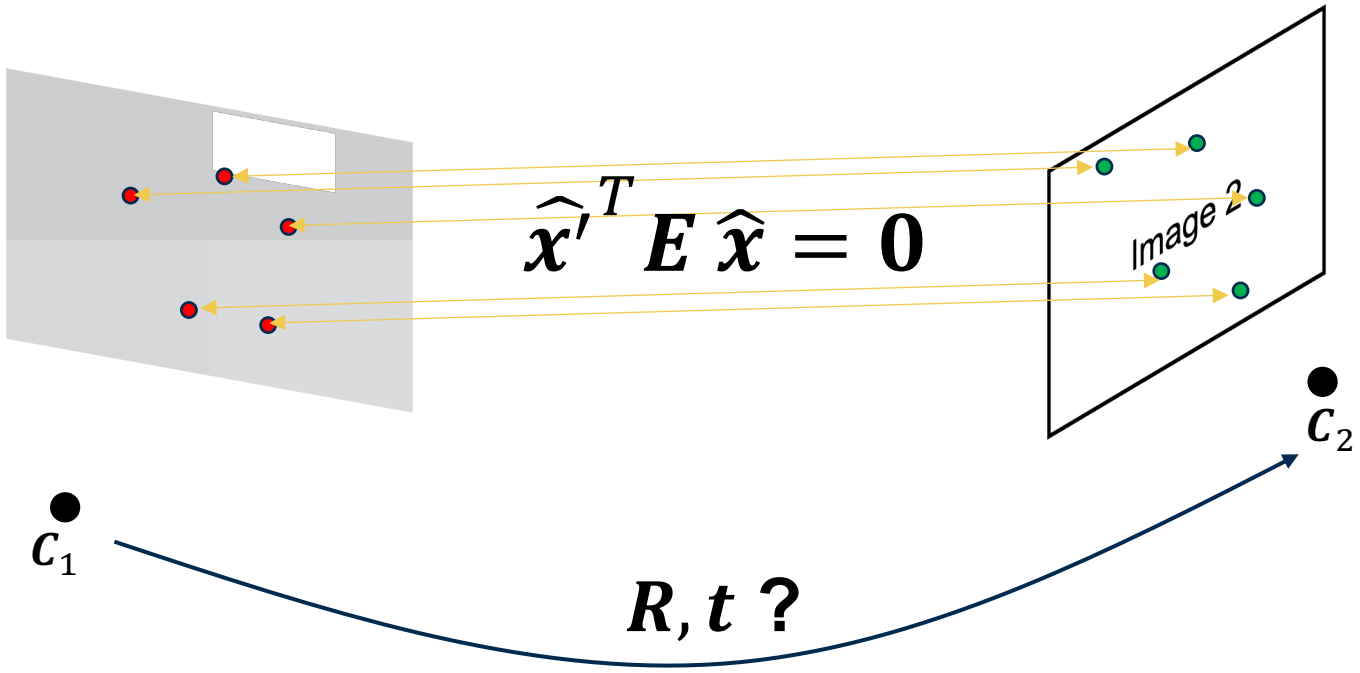
Summary: Motion from 2D-2D Matches



- Find interest points in both images.
- Match them.

CRICOS No.00213J

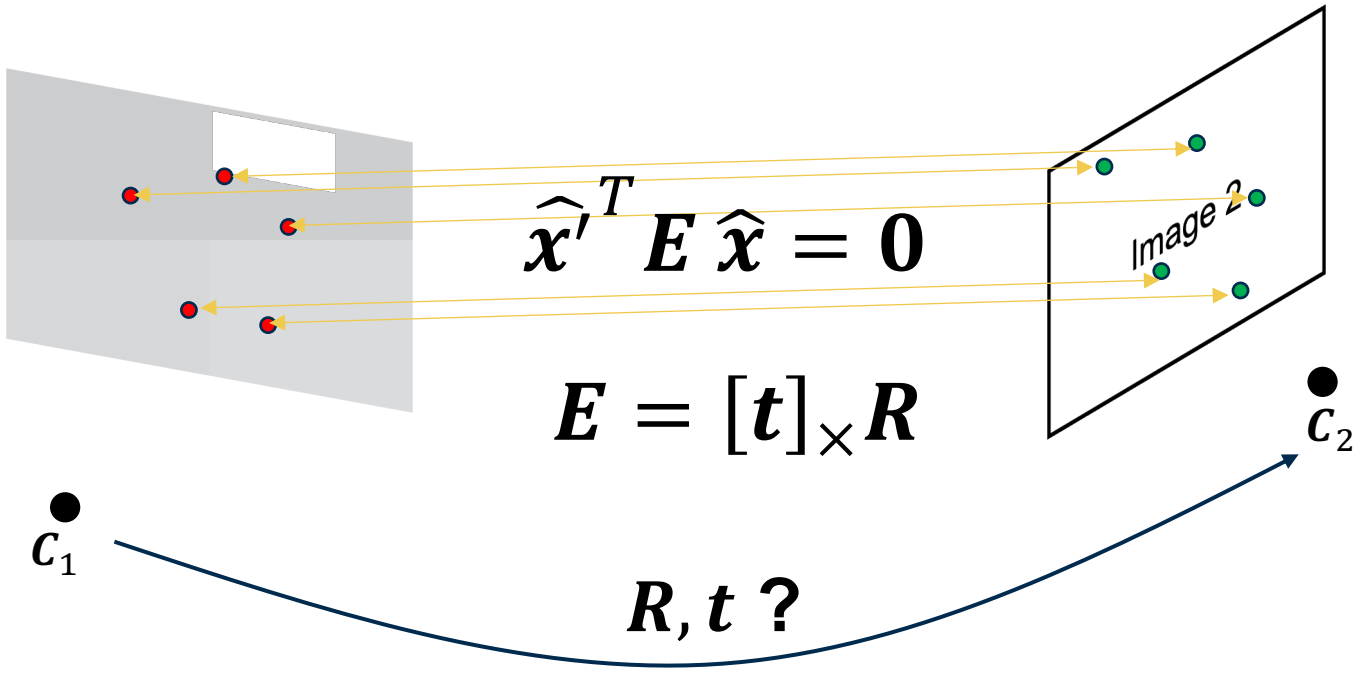
Summary: Motion from 2D-2D Matches



- Find interest points in both images.
- Match them.
- Calculate Essential Matrix using `findEssentialMat()`

CRICOS No.00213J

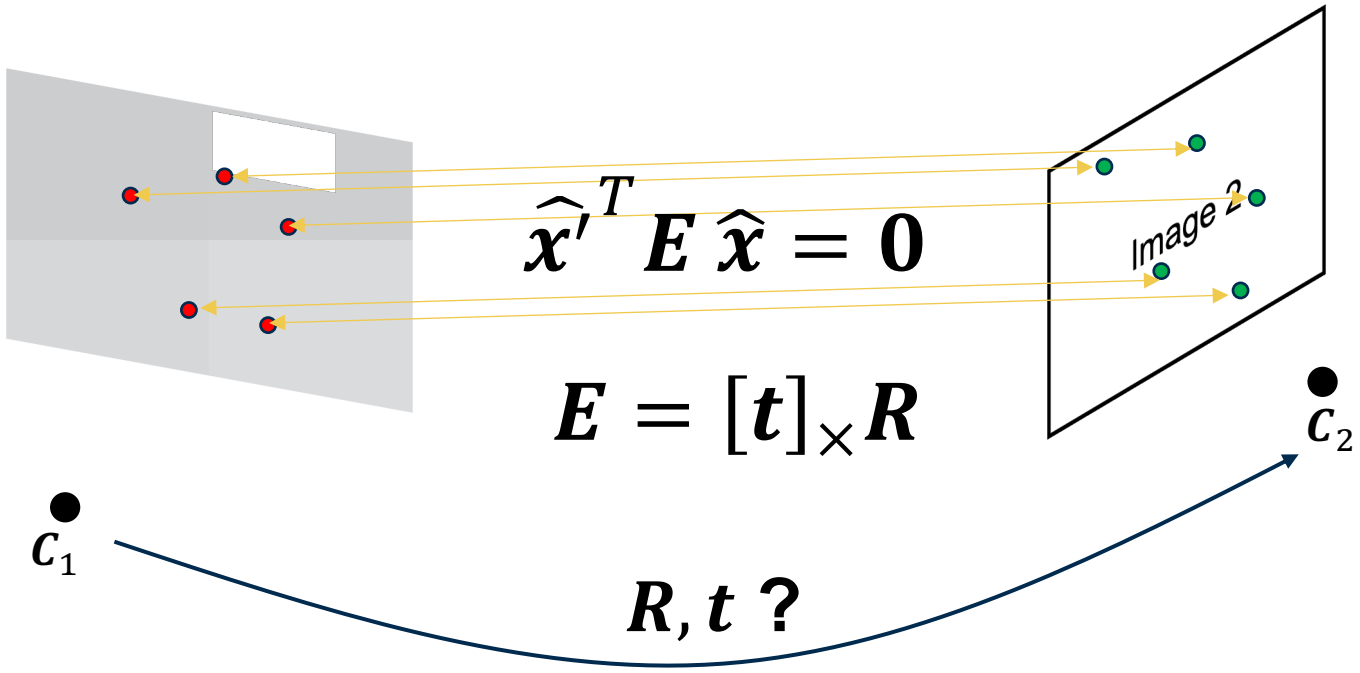
Summary: Motion from 2D-2D Matches



- Find interest points in both images.
- Match them.
- Calculate Essential Matrix using `findEssentialMat()`
- Decompose into **R** and **t** using `decomposeEssentialMat()`
- Find correct solution with `trianglulatePoints()`

CRICOS No.00213J

Summary: Motion from 2D-2D Matches



- Find interest points in both images.
- Match them.
- Calculate Essential Matrix using `findEssentialMat()`
- ~~Decompose into **R** and **t** using `decomposeEssentialMat()`~~
- ~~Find correct solution with `triangulatePoints()`~~

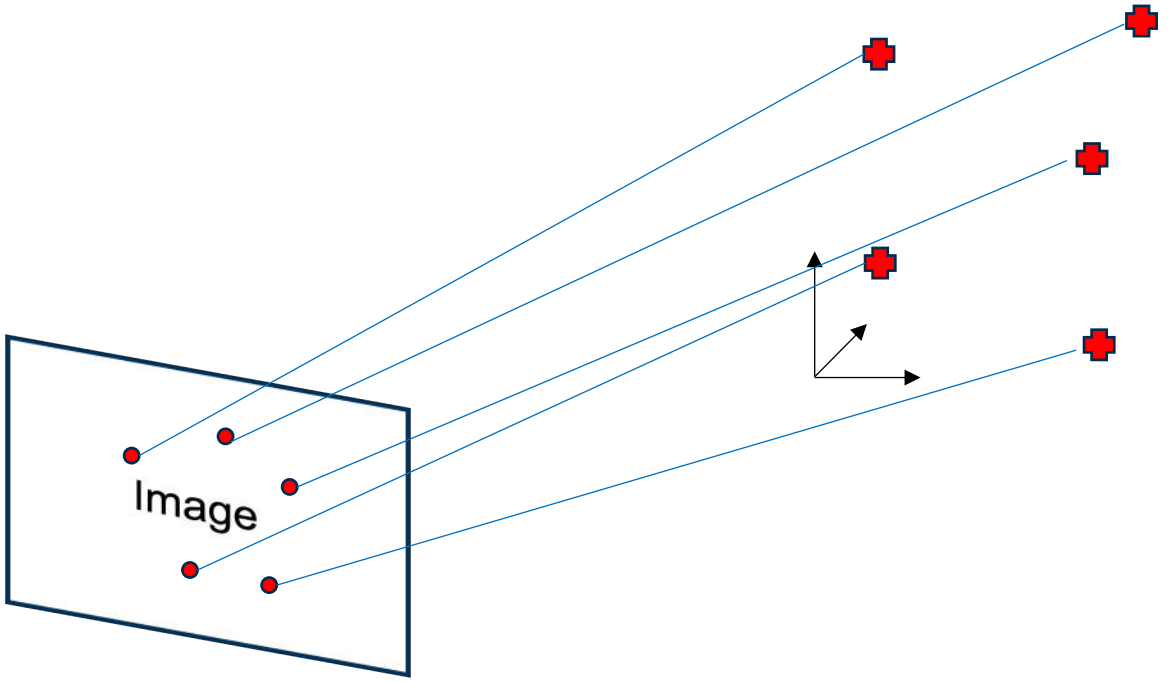
- Find **R** and **t** using `recoverPose(E, points1, points2)`

CRICOS No.00213J



Outlook

3D-2D Correspondences



3D-3D Correspondences

